Molecular dynamics coding exercises

Chi Y. Cheng, Maciej Bartkowiak, Sanghamitra Mukhopadhyay

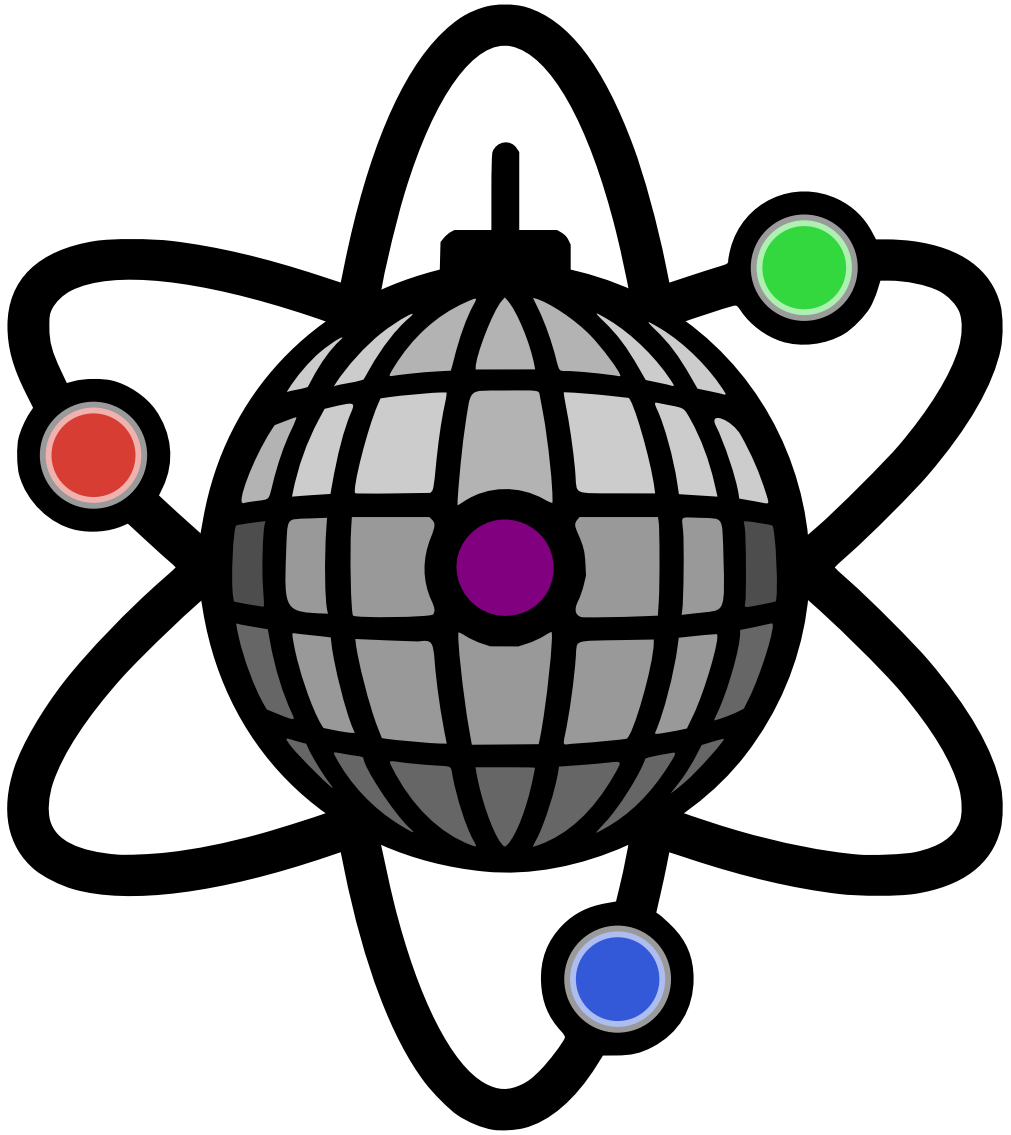
MDANSE Workshop 2026

03/11/2026



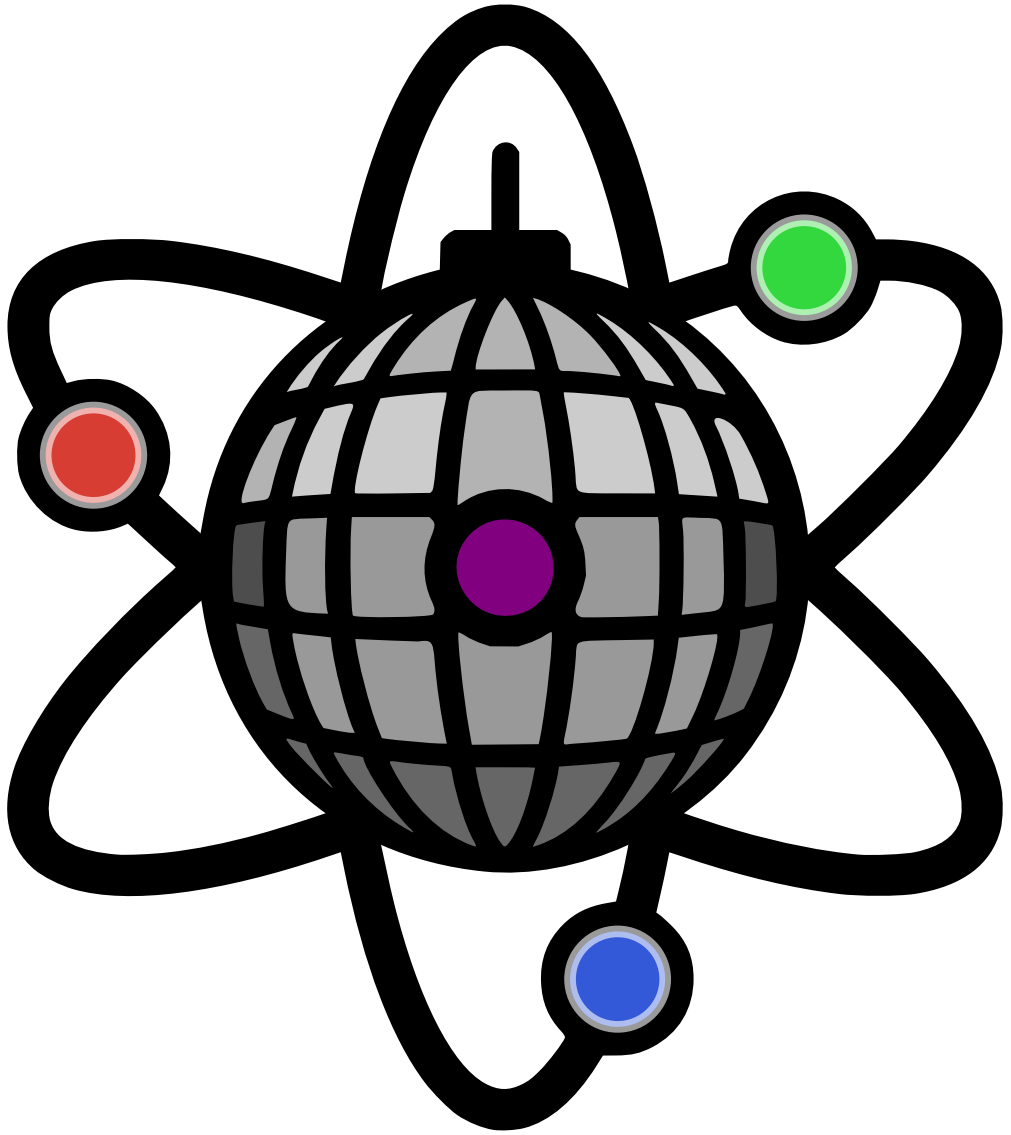
MD coding exercises

1. Single atom
2. Free particles
3. Simple diatomic
4. Lennard-Jones
5. Energies
6. Temperature
7. Pressure



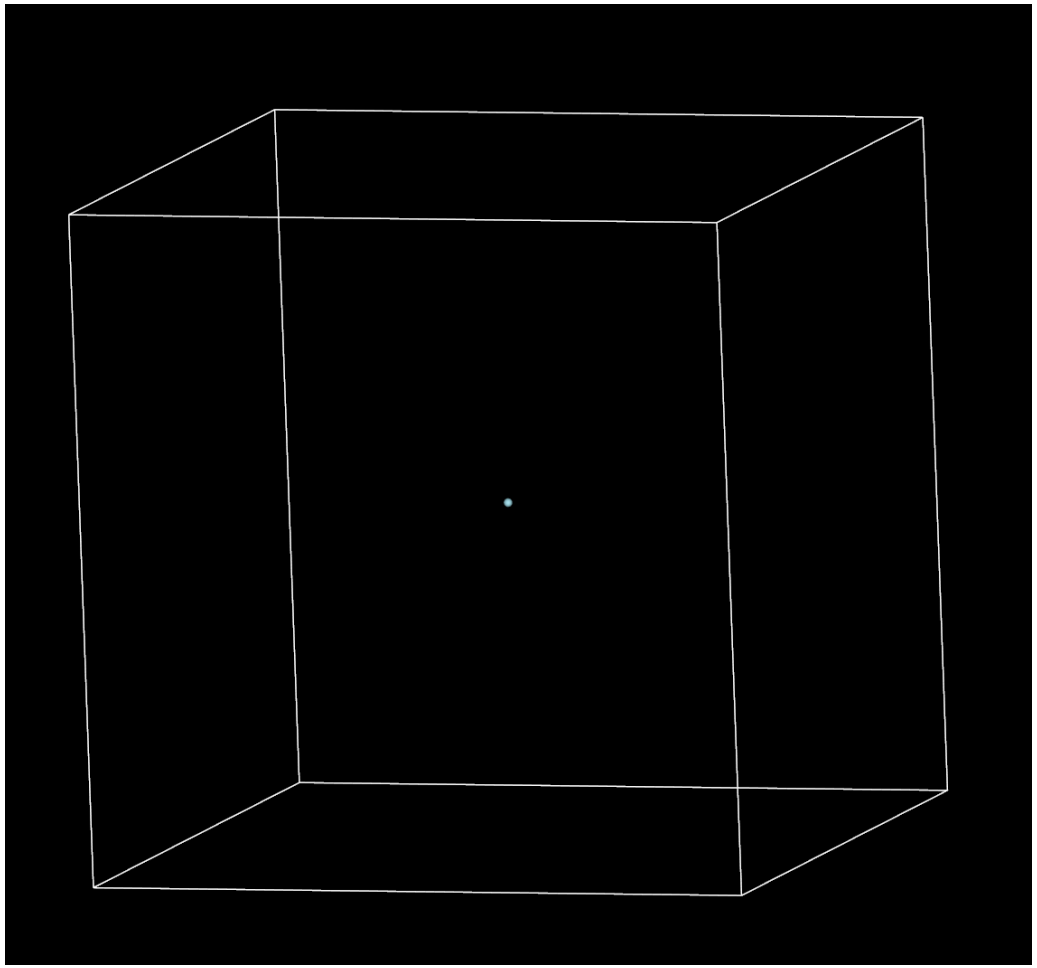
But why?

- You would never need to write your own MD package so why do this?
- The aim of the series of exercises is to apply some of the MD theory we have talked about, go through some more practical aspect of MD like periodic boundary conditions, and learn at bit of coding as well.
- This should give you a more hands-on experience of MD theory.
- We don’t expect you to finish all tutorials, feel free to finish them at your own pace.



1. Single atom

- In this tutorial we will install MDANSE, generate a trajectory of a single atom with a constant velocity and then use MDANSE to visualise the trajectory.

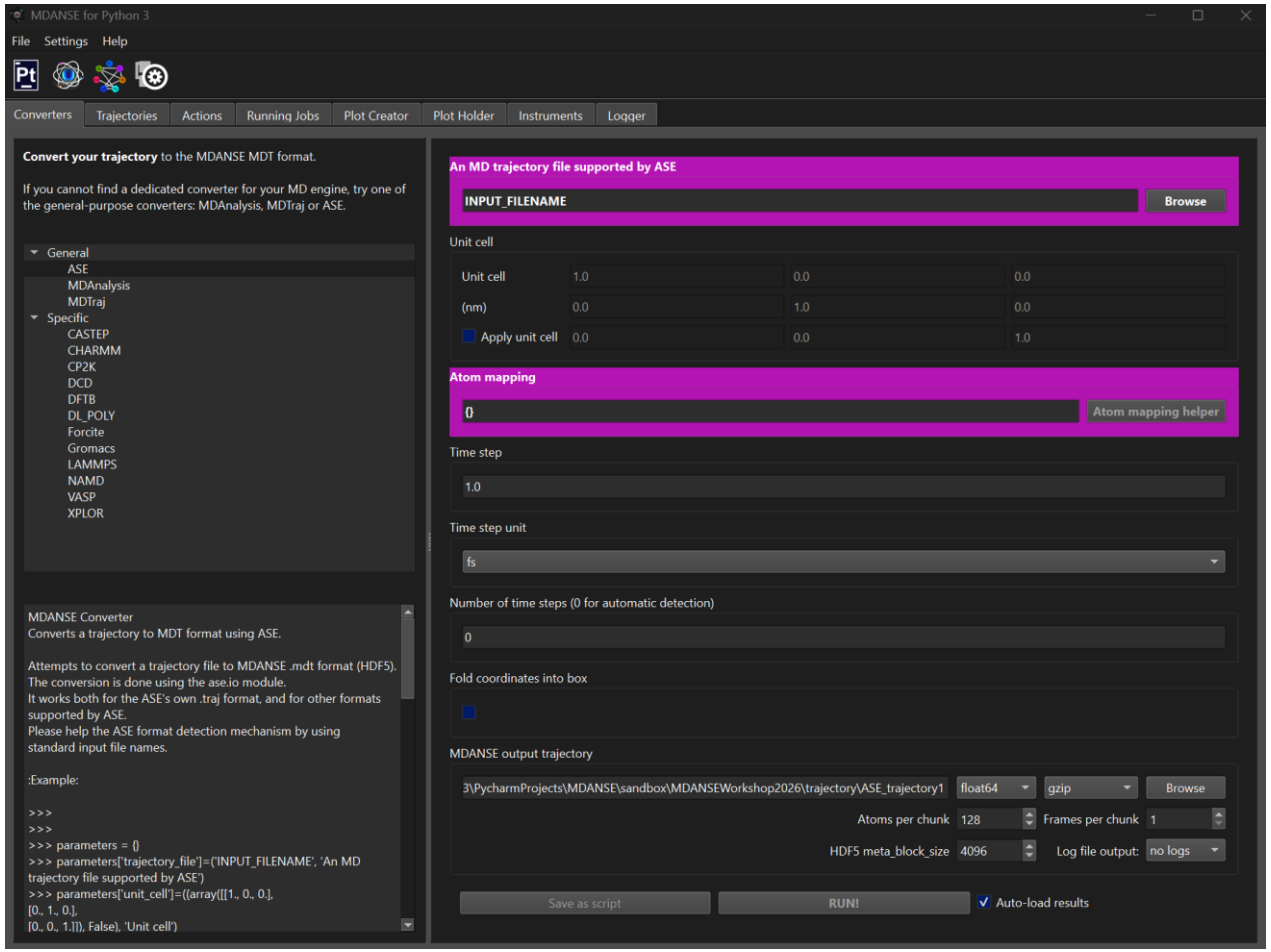


1. Single atom

- First let's install MDANSE if you have not already done so.
- Run the following command in the command line.

```
conda create -n mdanse python=3.13
conda activate mdanse
pip install mdanse[all]
pip install mdanse_gui[all]
mdanse_gui
```

- Note that it takes a bit of time for the `mdanse_gui` to load, give it a minute.



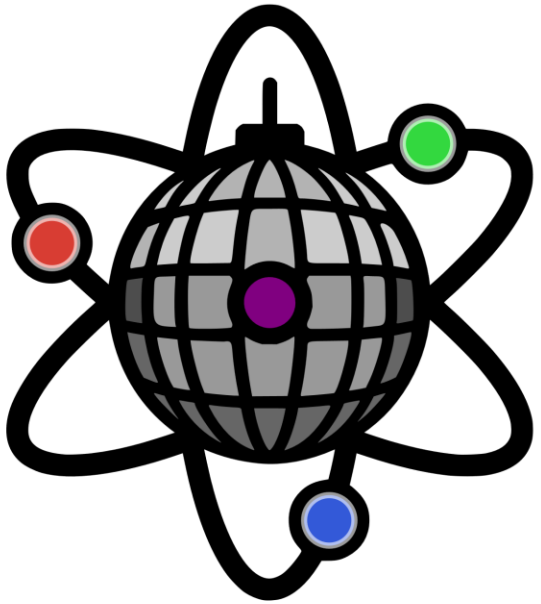
1. Single atom

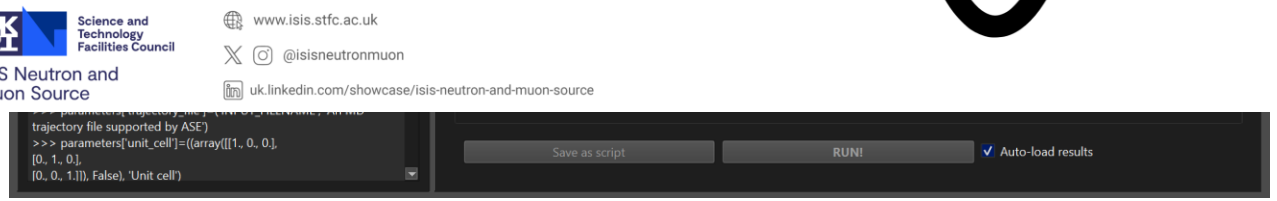
- If you are having issues, take a look at the troubleshooting slides.



Troubleshooting

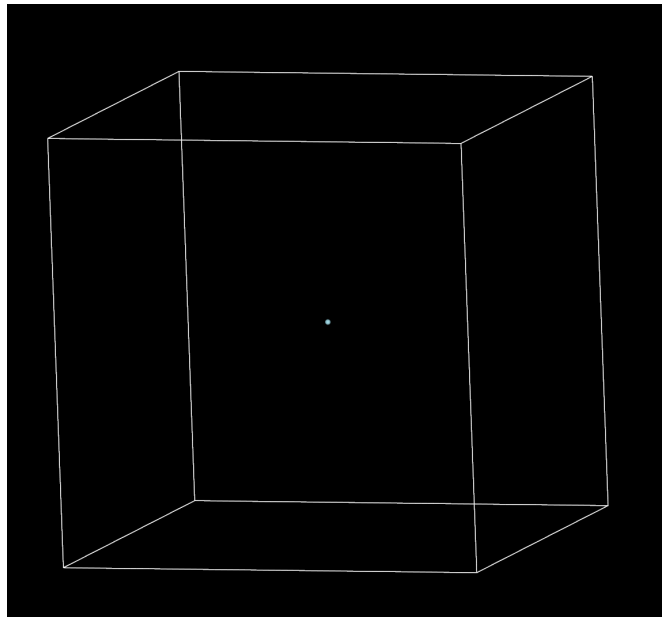
1. Troubleshooting – Windows 3D viewer slow or buggy





1. Single atom

- If the `mdanse_gui` ran fine then let's move the exercise scripts, starting with `single_atom.py`.



```python
import numpy as np


KB = 8.314462618e-3  # AMU nm^2 ps^-2 K^-1 = kJ mol^-1 K^-1
rng = np.random.default_rng()


def next_positions(pos: np.ndarray, vels: np.ndarray, ds: float) -> np.ndarray:
    """Returns the next set of positions after an amount of time ds has
    elapsed.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    ds : float
        The time step.

    Returns
    -------
    np.ndarray
        Next position after an amount of time ds has elapsed.
    """
    pass


def write_positions(pos: np.ndarray, b_len: float, file_name: str):  1 usage
    """Append positions to xyz file.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    b_len : float
        The simulation box length.
    file_name : str
        File append positions to.

    """
    with open(file_name, "a") as f:
        f.write(f"1\n")
        f.write(f'Lattice="{10 * b_len} 0.0 0.0 0.0 {10 * b_len} 0.0 0.0 0.0 {10 * b_len}" pbc="T T T"\n')
        f.write(f"Ar {10 * pos[0]} {10 * pos[1]} {10 * pos[2]}\n")
```

1. Single atom

```python
import numpy as np


KB = 8.314462618e-3  # AMU nm^2 ps^-2 K^-1 = kJ mol^-1 K^-1
rng = np.random.default_rng()


def next_positions(pos: np.ndarray, vels: np.ndarray, ds: float) -> np.ndarray:
    """Returns the next set of positions after an amount of time ds has
    elapsed.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    ds : float
        The time step.

    Returns
    -------
    np.ndarray
        Next position after an amount of time ds has elapsed.
    """
    pass
```

- We will use AMU (atomic mass units), nm, ps, and K units for all examples.

1. Single atom

```
48  # numerical simulation o      a single atom
49  box_len = 4.585  # nm
50  positions = box_len * np.array([0.5, 0.5, 0.5])
51  velocities = np.random.rand(3)  # nm ps-1
52  time_step = 0.01  # 0.01 ps = 10 fs
53  num_time_steps = 100
54
55
56  open("single_atom.xyz", "w").close()  # clear the file
57  for _ in range(num_time_steps):
58      write_positions(positions, box_len, "single_atom.xyz")
59      positions = next_positions(positions, velocities, time_step)
60
61
62  # to check your results you can convert this xyz file using the ASE
63  # converter in MDANSE.
```

- Here we set up some of the initial parameter for our simulation such as the box size and time step.

1. Single atom

```
48  # numerical simulation of system of a single atom
49  box_len = 4.585  # nm
50  positions = box_len * np.array([0.5, 0.5, 0.5])
51  velocities = np.random.rand(3)  # nm ps^-1
52  time_step = 0.01  # 0.01 ps = 10 fs
53  num_time_steps = 100
54
55
56  open("single_atom.xyz", "w").close()  # clear the file
57  for _ in range(num_time_steps):
58      write_positions(positions, box_len, "single_atom.xyz")
59      positions = next_positions(positions, velocities, time_step)
60
61
62  # to check your results you can convert this xyz file using the ASE
63  # converter in MDANSE.
```

- Here we set up some of the initial parameter for our simulation such as the box size and time step.
- In MD we need to input some initial coordinates, this could come from some experimental structure for example. Since we are modelling a single atom, we will set it to the centre of the box.

1. Single atom

```
48  # numerical simulation of system of a single atom
49  box_len = 4.585  # nm
50  positions = box_len * np.array([0.5, 0.5, 0.5])
51  velocities = np.random.rand(3)  # nm ps^-1
52  time_step = 0.01  # 0.01 ps = 10 fs
53  num_time_steps = 100
54
55
56  open("single_atom.xyz", "w").close()  # clear the file
57  for _ in range(num_time_steps):
58      write_positions(positions, box_len, "single_atom.xyz")
59      positions = next_positions(positions, velocities, time_step)
60
61
62  # to check your results you can convert this xyz file using the ASE
63  # converter in MDANSE.
```



- Here we set up some of the initial parameter for our simulation such as the box size and time step.
- In MD we need to input some initial coordinates, this could come from some experimental structure for example. Since we are modelling a single atom, we will set it to the centre of the box.
- In this part of the code, we clear the output xyz file and begin writing the positions of our single atom.

1. Single atom

```
48  # numerical simulation of system of a single atom
49  box_len = 4.585  # nm
50  positions = box_len * np.array([0.5, 0.5, 0.5])
51  velocities = np.random.rand(3)  # nm ps^-1
52  time_step = 0.01  # 0.01 ps = 10 fs
53  num_time_steps = 100
54
55
56  open("single_atom.xyz", "w").close()  # clear the file
57  for _ in range(num_time_steps):
58      write_positions(positions, box_len, "single_atom.xyz")
59      positions = next_positions(positions, velocities, time_step)
60
61
62  # to check your results you can convert this xyz file using the ASE
63  # converter in MDANSE.
```


- Here we set up some of the initial parameter for our simulation such as the box size and time step.
- In MD we need to input some initial coordinates, this could come from some experimental structure for example. Since we are modelling a single atom, we will set it to the centre of the box.
- In this part of the code, we clear the output xyz file and begin writing the positions of our single atom.
- Here we advance the system one time step and get the new positions.

1. Single atom

```python
import numpy as np

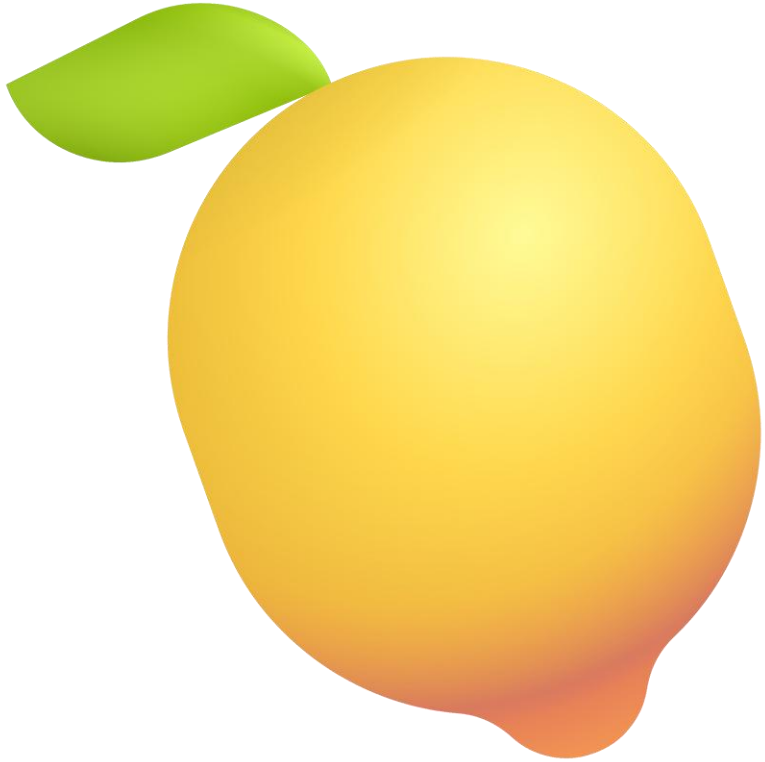

KB = 8.314462618e-3  # AMU nm^2 ps^-2 K^-1 = kJ mol^-1 K^-1
rng = np.random.default_rng()


def next_positions(pos: np.ndarray, vels: np.ndarray, ds: float) -> np.ndarray:
    """Returns the next set of positions after an amount of time ds has
    elapsed.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    ds : float
        The time step.

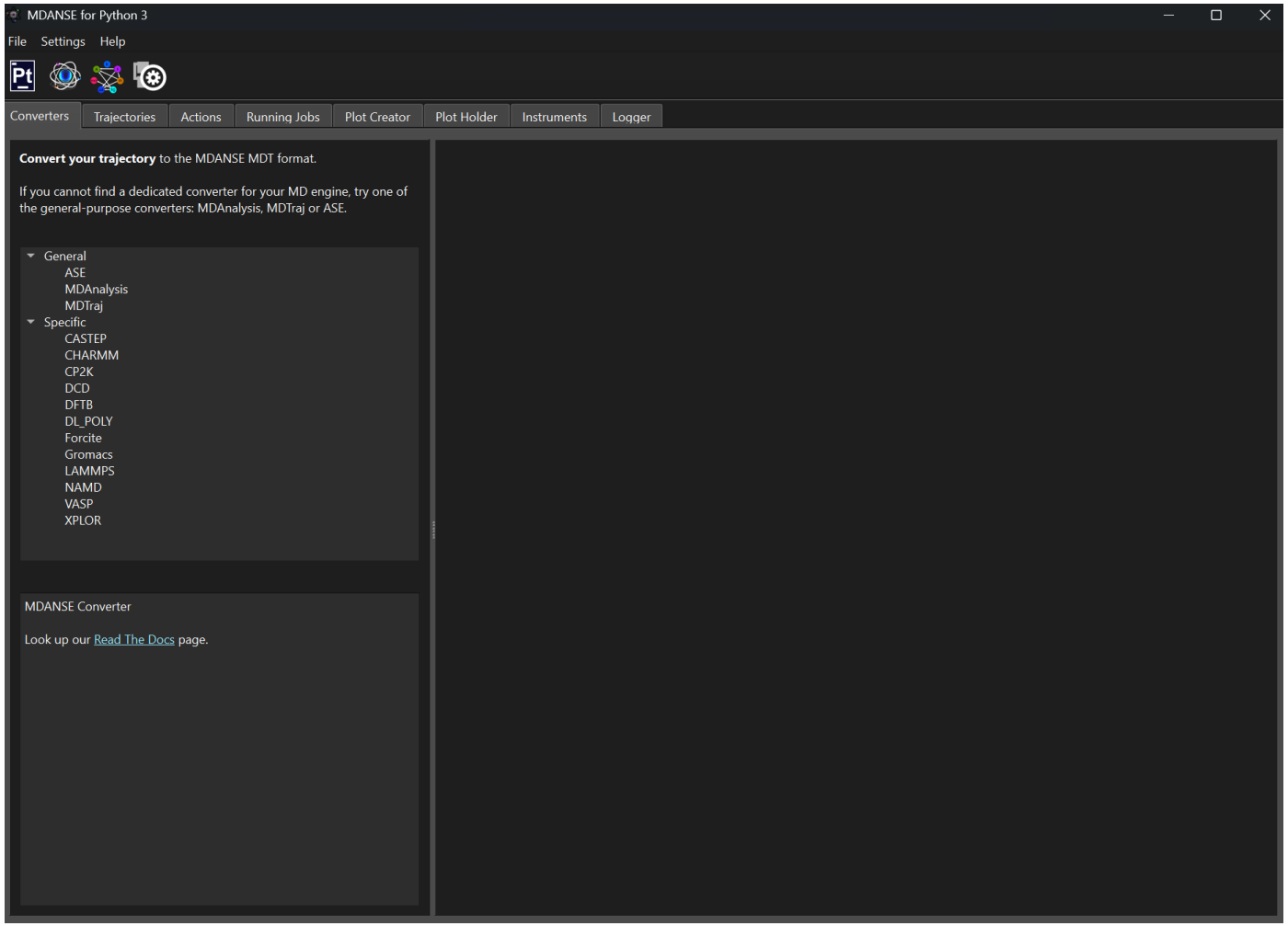
    Returns
    -------
    np.ndarray
        Next positions after an amount of time ds has elapsed.
    """
    pass
```

- When you see `pass` try to fill in the code. The docstring should provide enough information for you to figure out what needs to be returned.
- In this function we calculate the next position from the current position and velocity after an amount of time ds has elapsed.
- Since its just a single atom there are no forces!



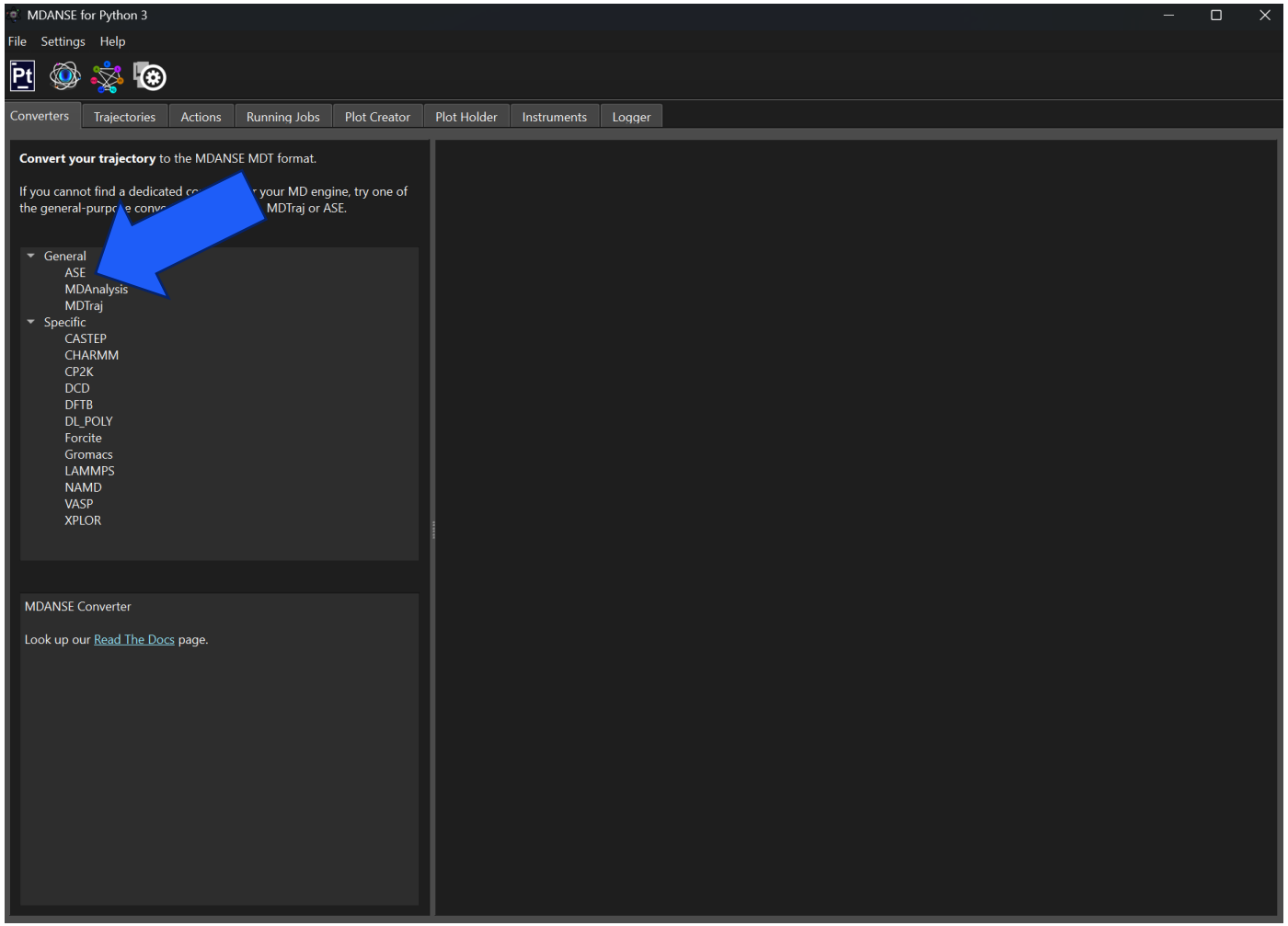
1. Single atom

- Once you have completed the script run it with python. This will produce an output file in an extended XYZ file format.
- Extended xyz files contain atom positions and can also include unit cell information in addition to several other things.
- This can be converted into MDANSE format and view in the MDANSE GUI.



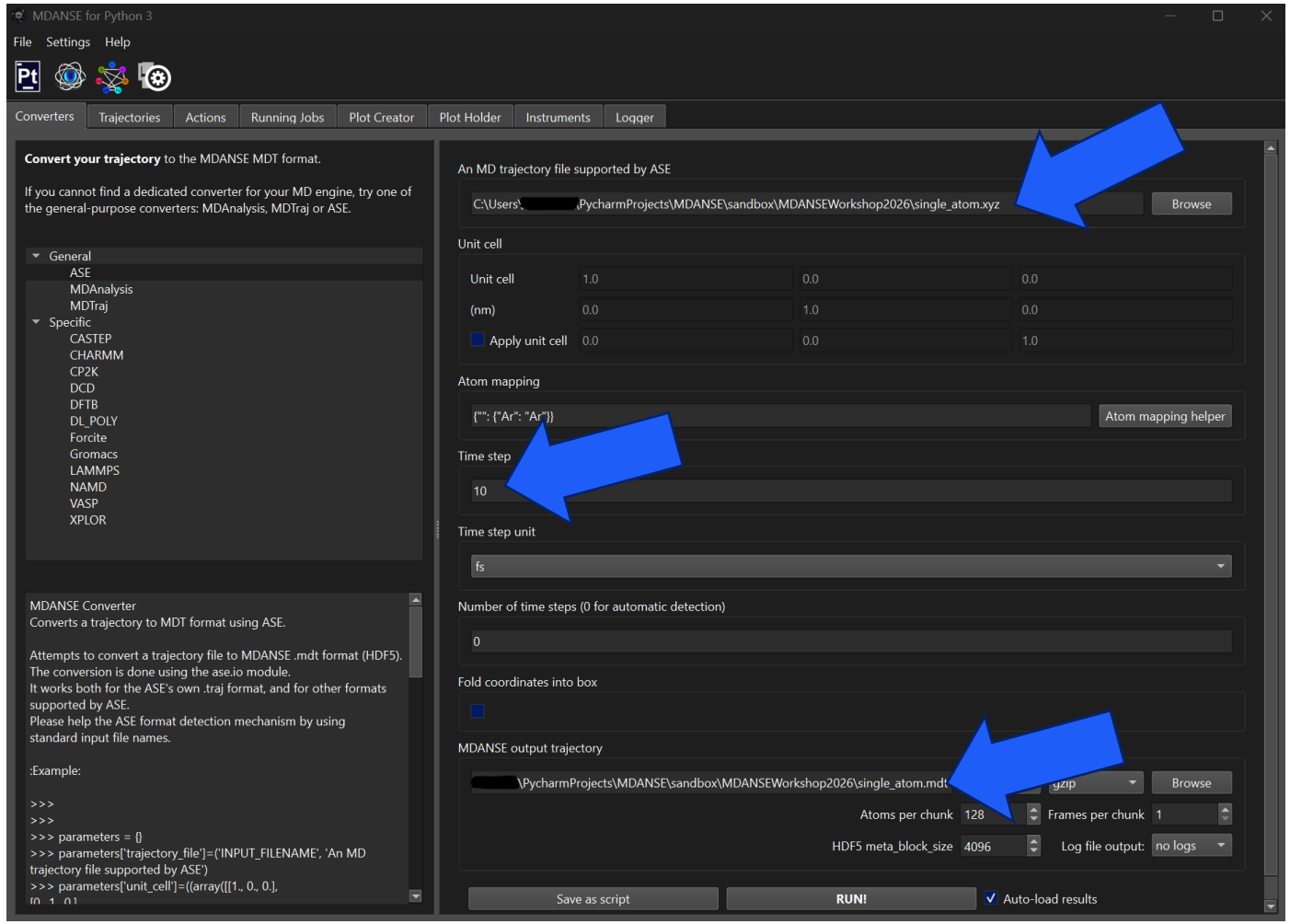
1. Single atom

- In the GUI select the ASE converter.



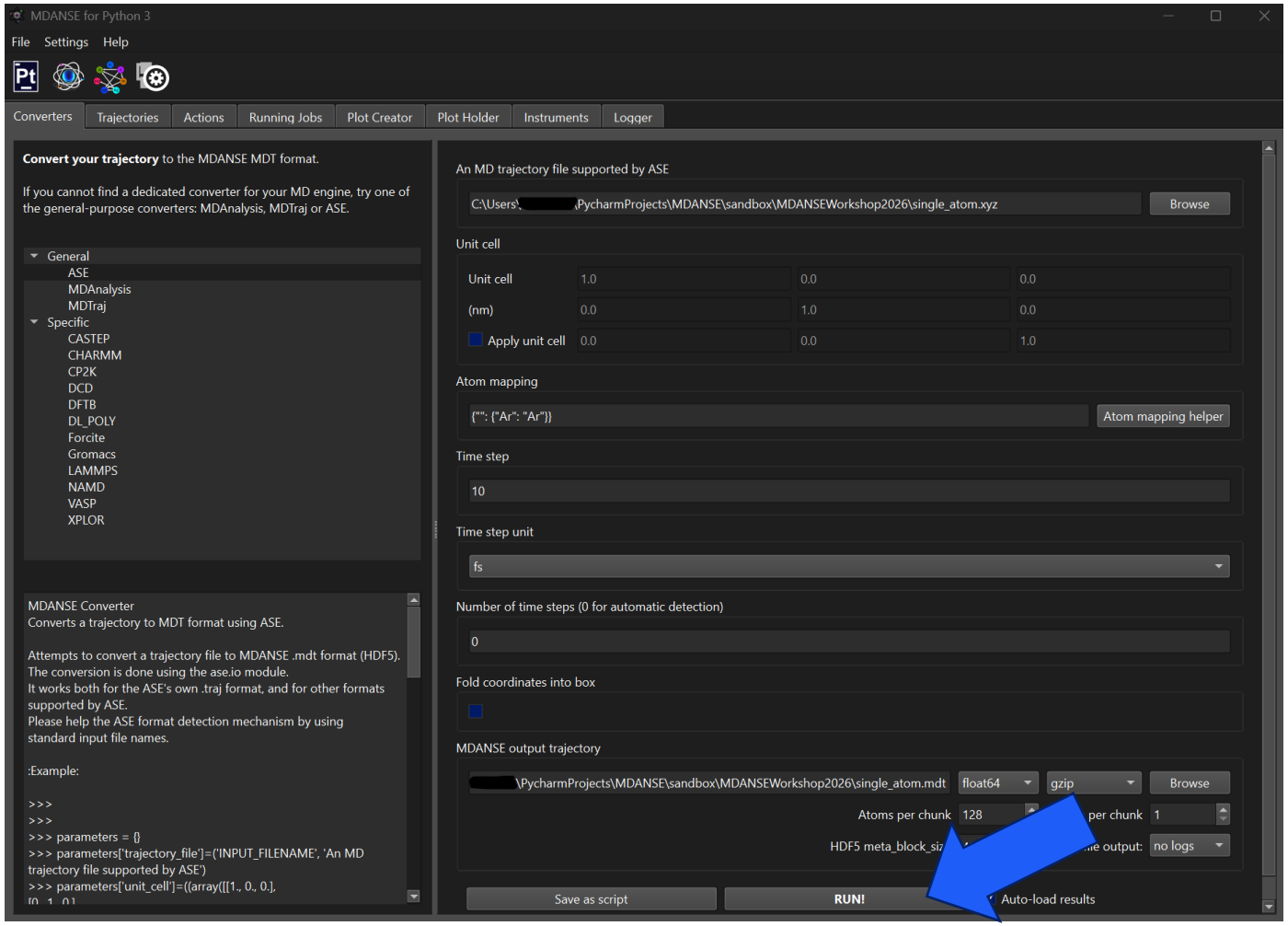
1. Single atom

- In the GUI select the ASE converter.
- Fill in the input file, the time step from the script, and the output file name.



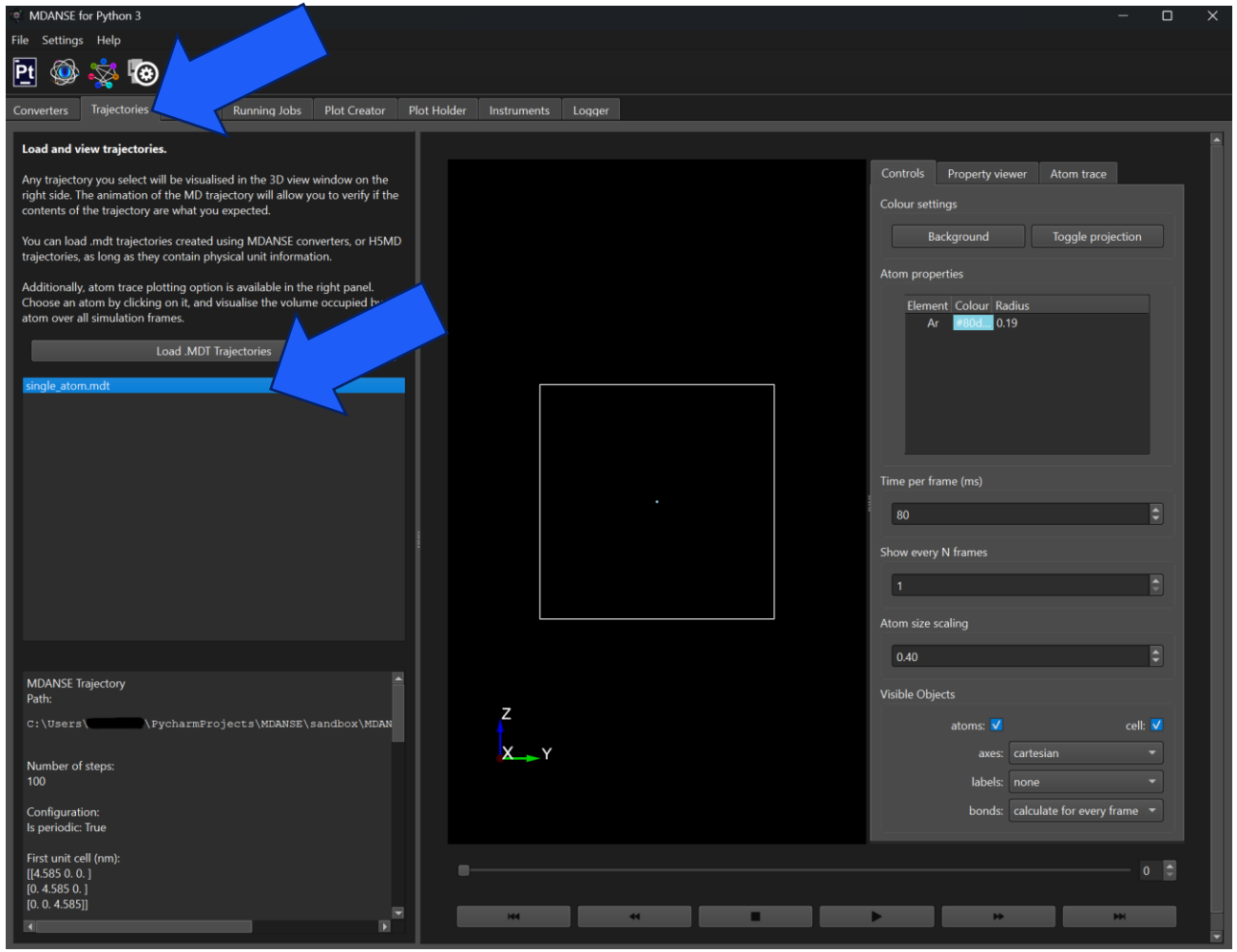
1. Single atom

- In the GUI select the ASE converter.
- Fill in the input file, the time step from the script, and the output file name.
- Once done, hit run.



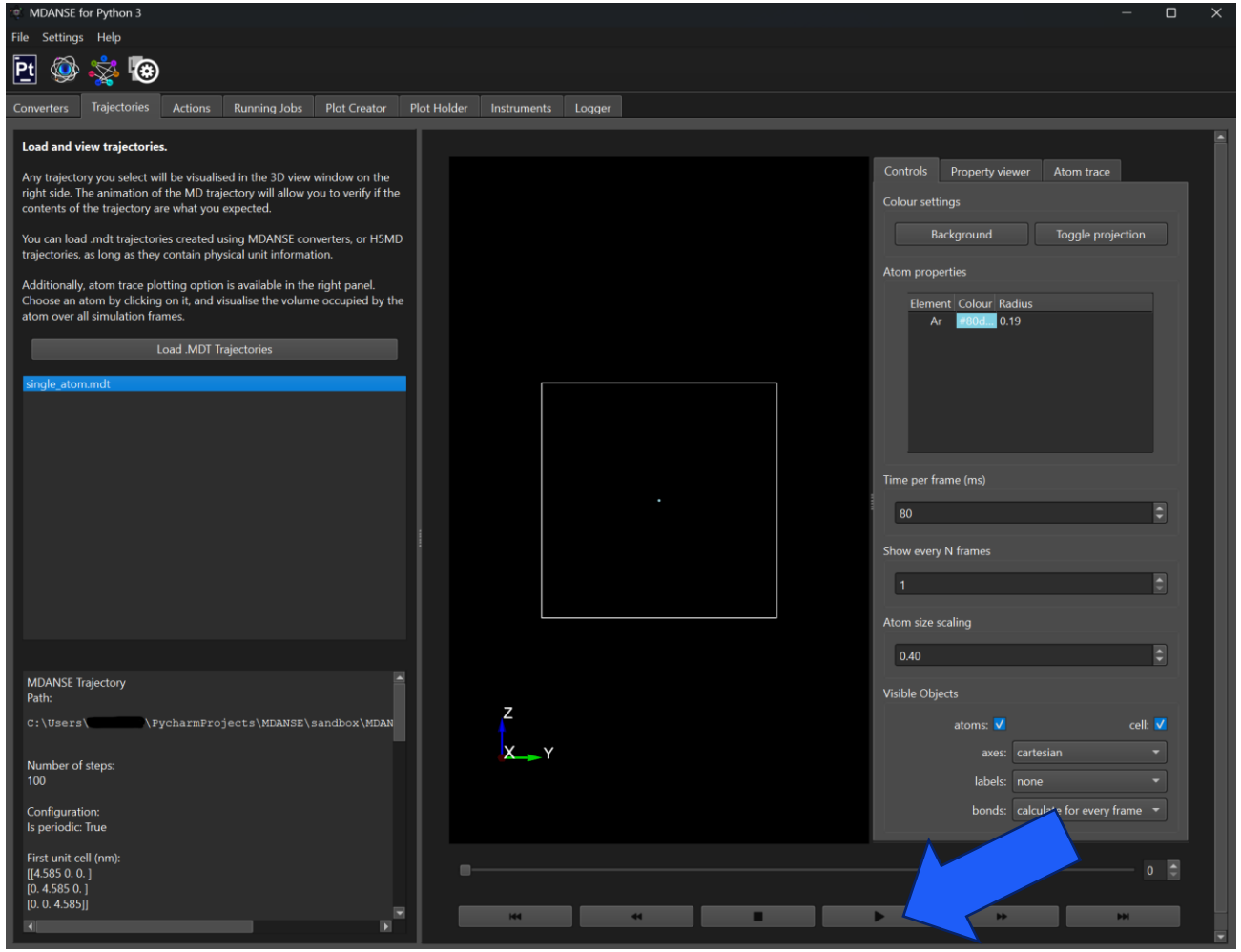
1. Single atom

- In the GUI select the ASE converter.
- Fill in the input file, the time step from the script, and the output file name.
- Once done, hit run.
- Next click the trajectories tab and then the single_atom.mdt trajectory or whatever you saved it as.



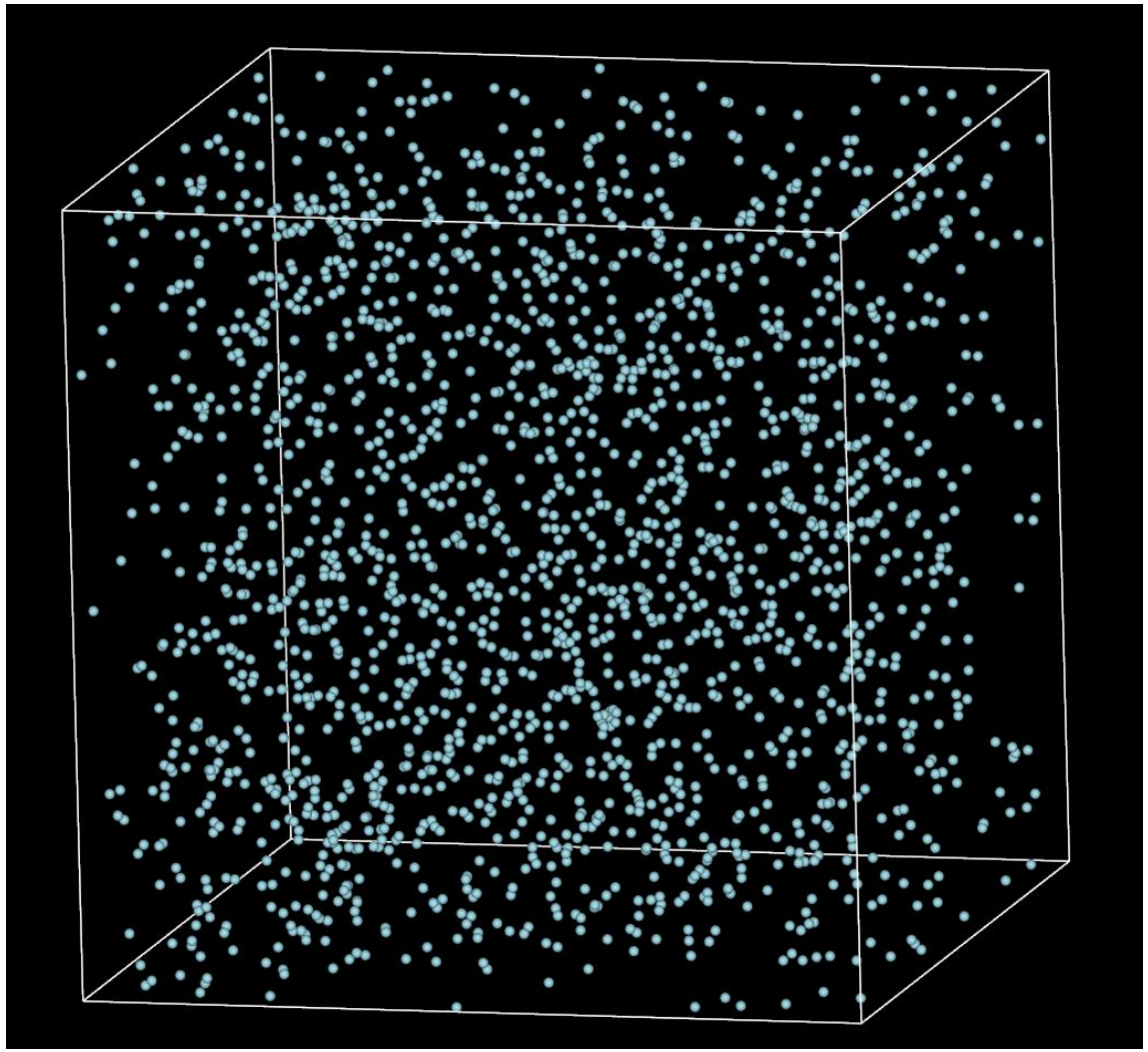
1. Single atom

- In the GUI select the ASE converter.
- Fill in the input file, the time step from the script, and the output file name.
- Once done, hit run.
- Next click the trajectories tab and then the single_atom.mdt trajectory or whatever you saved it as.
- To run the animation, press the play button. If you did everything right, you should see the atom move inside the box.



2. Free particles

- Probably one of the most basic systems that we can code up but still has some important features in all MD system such as positions, velocities, periodic boundary conditions, temperature, and time steps.



2. Free particles

```
90  # numerical simulation of system of free particles
91  mass = 39.948  # amu
92  box_len = 4.585  # nm
93  num_atoms = 2048
94  temperature = 85  # K
95  positions = box_len * np.random.rand(num_atoms, 3)
96  velocities = random_velocities_from_maxwell_boltzmann(num_atoms, temperature, mass)
97  time_step = 0.1  # 0.1 ps = 100 fs
98  num_time_steps = 1000
99
100
101 open("free_particles.xyz", "w").close()  # clear the file
102 for _ in range(num_time_steps):
103     write_positions(positions, box_len, "free_particles.xyz")
104     positions = next_positions(positions, velocities, time_step)
105     positions = wrap_coordinates(positions, box_len)
106
```

- As with the previous example, here we set some parameter but now we have a few more like temperature.

2. Free particles

```
90  # numerical simulation of system of free particles
91  mass = 39.948  # amu
92  box_len = 4.585  # nm
93  num_atoms = 2048
94  temperature = 85  # K
95  positions = box_len * np.random.rand(num_atoms, 3)
96  velocities = random_velocities_from_maxwell_boltzmann(num_atoms, temperature, mass)
97  time_step = 0.1  # 0.1 ps = 100 fs
98  num_time_steps = 1000
99
100
101 open("free_particles.xyz", "w").close()  # clear the file
102 for _ in range(num_time_steps):
103     write_positions(positions, box_len, "free_particles.xyz")
104     positions = next_positions(positions, velocities, time_step)
105     positions = wrap_coordinates(positions, box_len)
106
```

- As with the previous example, here we set some parameter but now we have a few more like temperature.
- Now we want to create a large number of atoms. To do this we create a 2D array, one xyz position for each atom. Since it's a system of free particles we will set there positions random in the simulation box.

2. Free particles

```
90  # numerical simulation of system of free particles
91  mass = 39.948  # amu
92  box_len = 4.585  # nm
93  num_atoms = 2048
94  temperature = 85  # K
95  positions = box_len * np.random.rand(num_atoms, 3)
96  velocities = random_velocities_from_maxwell_boltzmann(num_atoms, temperature, mass)
97  time_step = 0.1  # 0.1 ps = 100 fs
98  num_time_steps = 1000
99
100
101 open("free_particles.xyz", "w").close()  # clear the file
102 for _ in range(num_time_steps):
103     write_positions(positions, box_len, "free_particles.xyz")
104     positions = next_positions(positions, velocities, time_step)
105     positions = wrap_coordinates(positions, box_len)
106
```

- As with the previous example, here we set some parameter but now we have a few more like temperature.
- Now we want to create a large number of atoms. To do this we create a 2D array, one xyz position for each atom. Since it's a system of free particles we will set there positions random in the simulation box.
- Here we advance the system one time step and get the new positions as done previously, but at the end we wrap the coordinates so all atoms are in the simulation box.

2. Free particles

```python
def random_velocities_from_maxwell_boltzmann(n_atms: int, temp: float, mass: float) -> np.ndarray:
    """Generate velocities whose speeds are drawn from a maxwell boltzmann
    distribution.

    Parameters
    ----------
    n_atms : int
        Number of atoms to generate velocities for.
    temp : float
        Temperature in kelvin.
    mass : float
        Mass of the particles.

    Returns
    -------
    np.ndarray
        Numpy array of particle velocities drawn from maxwell boltzmann
        distribution.
    """
    pass

```

- In the previous slides we set the coordinates randomly, but MD also needs initial velocities.
- Generally, velocities are set by sampling them from a Maxwell-Boltzmann distributions at a specific temperature that you want the simulation to run at.
- The Maxwell-Boltzmann distribution for a velocity vector is:

$$f(v_x, v_y, v_z) = \left(\frac{m}{2\pi k_\mathrm{B} T}\right)^{3/2} \exp\left[-\frac{m(v_x^2 + v_y^2 + v_z^2)}{2k_\mathrm{B}T}\right]$$

2. Free particles

```python
def next_positions(pos: np.ndarray, vels: np.ndarray, ds: float) -> np.ndarray:
    """Returns the next set of positions after an amount of time ds has
    elapsed.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    ds : float
        The time step.

    Returns
    -------
    np.ndarray
        Next position after an amount of time ds has elapsed.
    """
    # copy from single atom
    pass
```

- In this function we calculate the next position from the current position and velocity after an amount of time ds has elapsed.
- Similarly to the single atom example, there are no forces acting on the particles.
- The code from the single particles should also work with our 2D arrays. We can just copy the code.

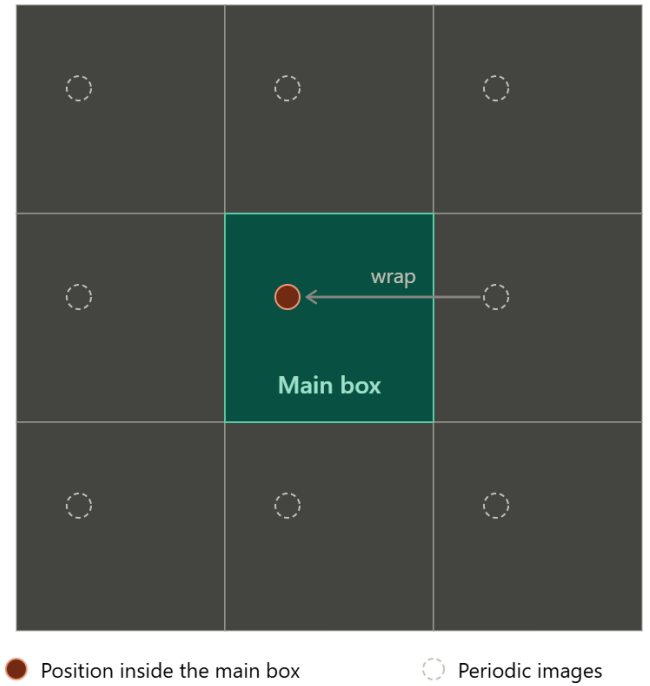
2. Free particles

```python
def wrap_coordinates(pos: np.ndarray, b_len: float) -> np.ndarray:
    """Folds the atom positions back into the simulation box.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    b_len : float
        The simulation box length.

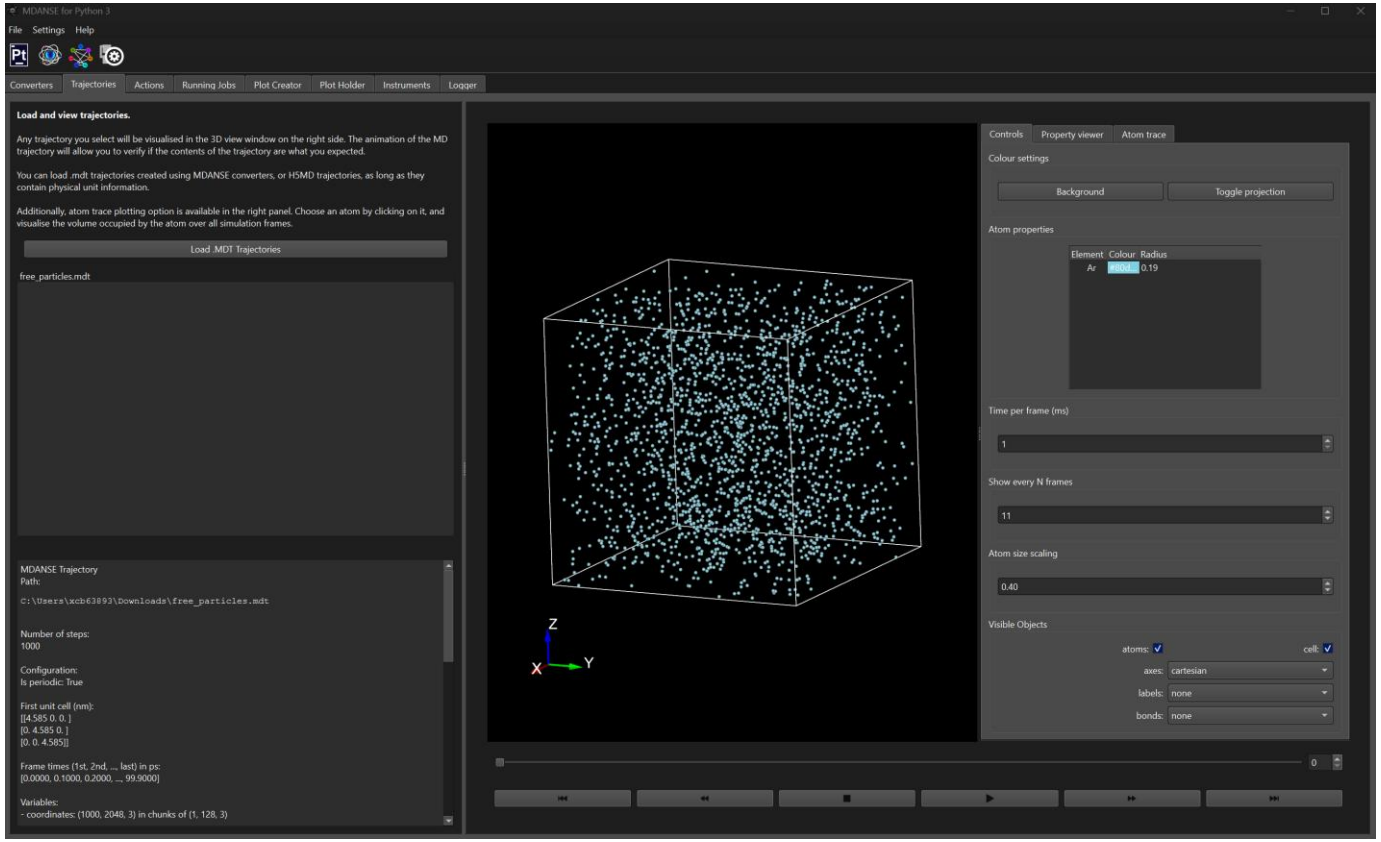
    Returns
    -------
    np.ndarray
        Atom positions folded into the simulation box.
    """
    pass
```

- Mostly important for visualisation purposes in our case, since the free particles will fly off to infinity this function wrap the coordinates back into the simulation box.
- In MD, all atoms have periodic images that are repeated to infinity. This function just says that we want the image that is within the “main” simulation box.



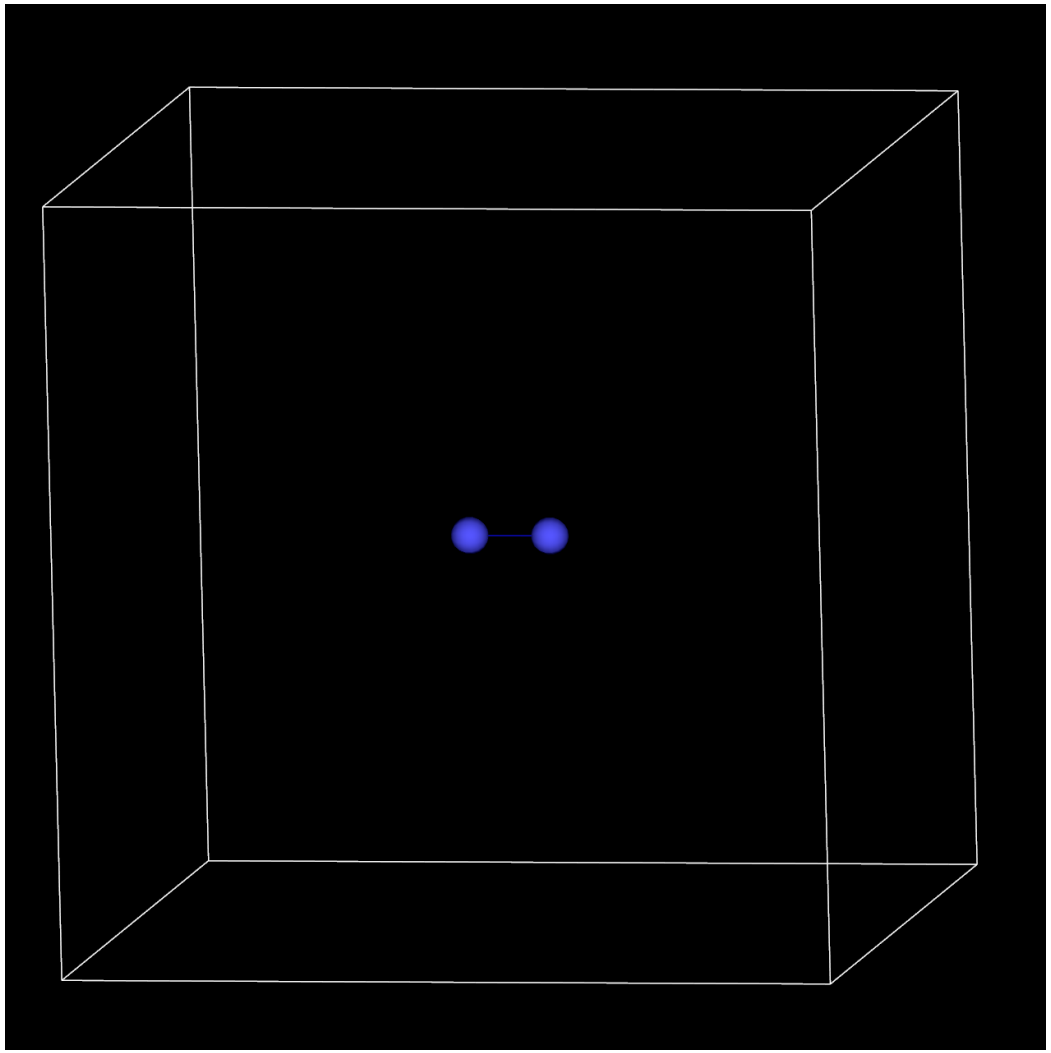
2. Free particles

- If you have successfully finished this example, you should be able to run the script, and it should write an extended xyz file.
- Try converting the file using the ASE converter in MDANSE to visualise it and then try running some analysis calculations on it.
- Make sure you put in the correct units and time step.



3. Simple diatomic

- In this exercise, we will model a single nitrogen diatomic.
- The potential between the two atoms will be modelled with a harmonic potential.
- As there is a force acting between the two atoms, the equations to get the positions at the next time step will be more complicated.



3. Simple diatomic

```python
def random_velocities_from_maxwell_boltzmann(n_atms: int, temp: float, mass: float) -> np.ndarray:
    """Generate velocities whose speeds are drawn from a maxwell boltzmann
    distribution.

    Parameters
    ----------
    n_atms : int
        Number of atoms to generate velocities for.
    temp : float
        Temperature in kelvin.
    mass : float
        Mass of the particles.

    Returns
    -------
    np.ndarray
        Numpy array of particle velocities drawn from maxwell boltzmann
        distribution.
    """
    # copy from free particles
    pass
```

- For these function copy the results from the previous exercise.

```python
def wrap_coordinates(pos: np.ndarray, b_size: float) -> np.ndarray:
    """Folds the atom positions back into the simulation box.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    b_size : float
        The simulation box size.

    Returns
    -------
    np.ndarray
        Atom positions folded into the simulation box.
    """
    # copy from free particles
    pass
```

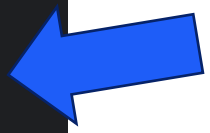
3. Simple diatomic

```
131  open("simple_diatomic.xyz", "w").close()  # clear the file
132  acceleration = calculate_acceleration(positions, mass, eq_bond_len, force_constant)
133  positions_0 = positions
134  write_positions(positions_0, box_len, "simple_diatomic.xyz")
135  positions_1 = positions + velocities * time_step + 0.5 * acceleration * time_step**2
136
137  for i in range(num_time_steps):
138      wrapped = wrap_coordinates(positions_1, box_len)
139      write_positions(wrapped, box_len, "simple_diatomic.xyz")
140
141      # verlet
142      acceleration = calculate_acceleration(positions_1, mass, eq_bond_len, force_constant)
143      positions_2 = next_positions(positions_0, positions_1, acceleration, time_step)
144      positions_0 = positions_1
145      positions_1 = positions_2
146
```

- To advance the simulation forward we will be using the Verlet algorithm.

3. Simple diatomic

```
131  open("simple_diatomic.xyz", "w").close()  # clear the file
132  acceleration = calculate_acceleration(positions, mass, eq_bond_len, force_constant)
133  positions_0 = positions
134  write_positions(positions_0, box_len, "simple_diatomic.xyz")
135  positions_1 = positions + velocities * time_step + 0.5 * acceleration * time_step**2
136
137  for i in range(num_time_steps):
138      wrapped = wrap_coordinates(positions_1, box_len)
139      write_positions(wrapped, box_len, "simple_diatomic.xyz")
140
141      # verlet
142      acceleration = calculate_acceleration(positions_1, mass, eq_bond_len, force_constant)
143      positions_2 = next_positions(positions_0, positions_1, acceleration, time_step)
144      positions_0 = positions_1
145      positions_1 = positions_2
146
```



- To advance the simulation forward we will be using the Verlet algorithm.
- In the first step of the Verlet algorithm, we calculate positions at the next time step t1.

$$\mathbf{r}_i(t_1) = \mathbf{r}_i(t_0) + \mathbf{v}_i(t_0)\Delta t + \frac{1}{2}\mathbf{a}_i(t_0)\Delta t^2$$

$$t_{n+1} = t_n + \Delta t$$

3. Simple diatomic

```
131  open("simple_diatomic.xyz", "w").close()  # clear the file
132  acceleration = calculate_acceleration(positions, mass, eq_bond_len, force_constant)
133  positions_0 = positions
134  write_positions(positions_0, box_len, "simple_diatomic.xyz")
135  positions_1 = positions + velocities * time_step + 0.5 * acceleration * time_step**2
136
137  for i in range(num_time_steps):
138      wrapped = wrap_coordinates(positions_1, box_len)
139      write_positions(wrapped, box_len, "simple_diatomic.xyz")
140
141      # verlet
142      acceleration = calculate_acceleration(positions_1, mass, eq_bond_len, force_constant)
143      positions_2 = next_positions(positions_0, positions_1, acceleration, time_step)
144      positions_0 = positions_1
145      positions_1 = positions_2
146
```

- To advance the simulation forward we will be using the Verlet algorithm.
- In the first step of the Verlet algorithm, we calculate positions at the next time step t1.

$$\mathbf{r}_i(t_1) = \mathbf{r}_i(t_0) + \mathbf{v}_i(t_0)\Delta t + \frac{1}{2}\mathbf{a}_i(t_0)\Delta t^2$$

$$t_{n+1} = t_n + \Delta t$$

- In this next steps we update the positions with acceleration and past positions.

$$\mathbf{r}_i(t_{n+1}) = 2\mathbf{r}_i(t_n) - \mathbf{r}_i(t_{n-1}) + \mathbf{a}_i(t_n)\Delta t^2$$

3. Simple diatomic

```python
def next_positions(pos: np.ndarray, vels: np.ndarray, acc: np.ndarray, ds: float) -> np.ndarray:
    """Returns the next positions.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    acc : np.ndarray
        Acceleration of the atoms in the simulation.
    ds : float
        The time step.

    Returns
    -------
    np.ndarray
        The next position.
    """
    pass
```

```python
def calculate_acceleration(pos: np.ndarray, mass: float, bond_eq: float, f_k: float):
    """Returns the acceleration on each atom.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    mass : float
        Mass of the atoms in the simulation.
    bond_eq : float
        The equilibrium bond length.
    f_k : float
        The force constant between the atoms in the diatomic.

    Returns
    -------
    np.ndarray
        Acceleration on each atom.
    """
    pass
```

$$\mathbf{r}_i(t_1) = \mathbf{r}_i(t_0) + \mathbf{v}_i(t_0)\Delta t + \frac{1}{2}\mathbf{a}_i(t_0)\Delta t^2$$

$$t_{n+1} = t_n + \Delta t$$

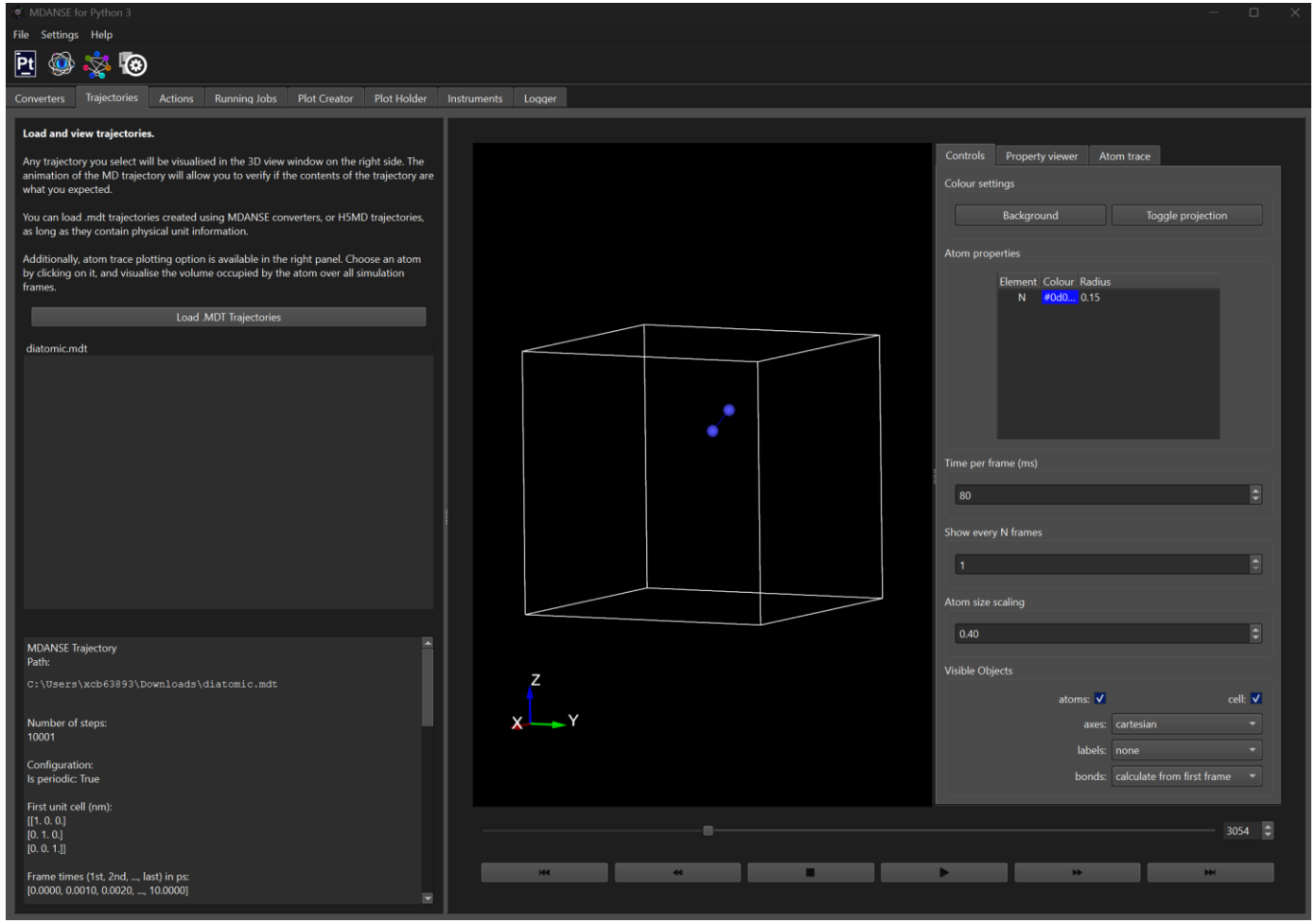
$$\mathbf{r}_i(t_{n+1}) = 2\mathbf{r}_i(t_n) - \mathbf{r}_i(t_{n-1}) + \mathbf{a}_i(t_n)\Delta t^2$$

- In the Verlet algorithm we will need the acceleration. We can determine this from the derivative of the potential.

$$F_x = -\frac{\partial V}{\partial x} \qquad V(\mathbf{r}_i, \mathbf{r}_j) = \frac{1}{2}k\left(|\mathbf{r}_i - \mathbf{r}_j| - r_{\text{eq}}\right)^2$$

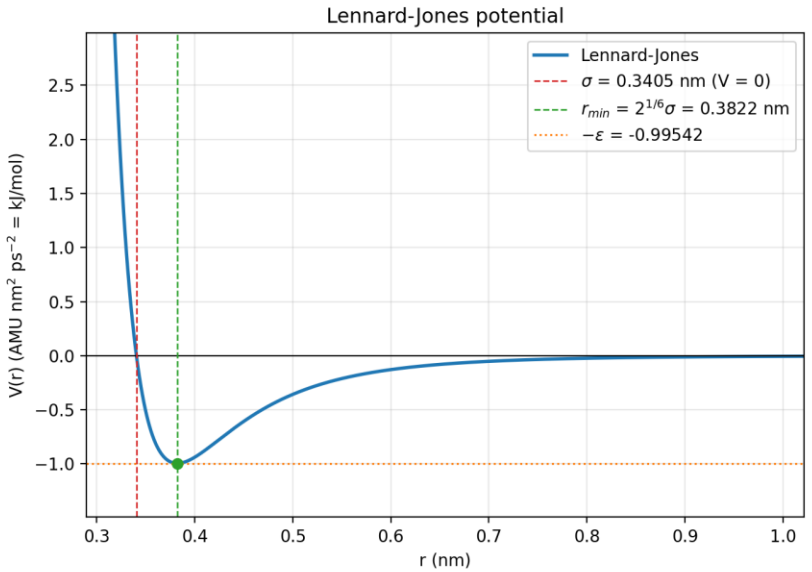
4. Lennard-Jones

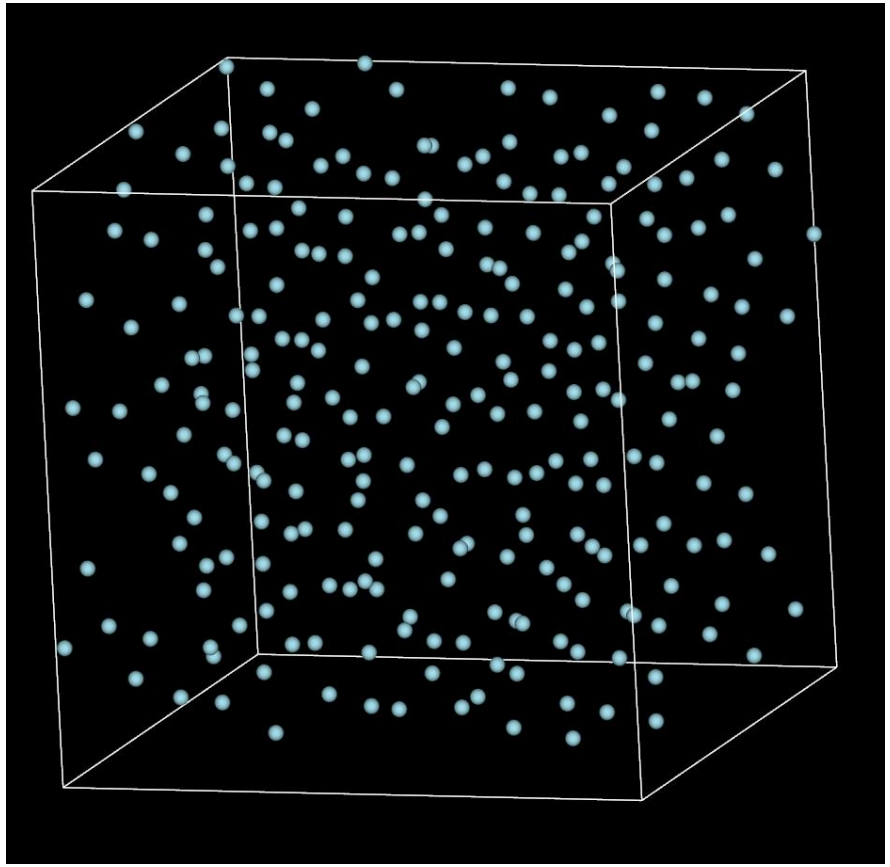
- If you have successfully finished this example, try converting the file using the ASE converter in MDANSE to visualise it.
- Try changing the force constant or some of the other parameters.



4. Lennard-Jones

- In this exercise will be model a system Lennard-Jones particles.
- There is a potential between all atoms in the system.





$$E = 4\varepsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^{6} \right]$$

4. Lennard-Jones

```python
# numerical simulation of a simple LJ system
mass = 39.948  # amu
n_cells = 4
cell_len = 0.57311 # nm
box_len = n_cells * cell_len  # nm
temperature = 85  # K
positions = cell_len * generate_fcc_lattice(n_cells)
num_atoms = positions.shape[0]
velocities = random_velocities_from_maxwell_boltzmann(num_atoms, temperature, mass)
time_step = 0.002  # 0.002 ps = 2 fs
num_time_steps = 5000
lj_sigma = 0.3405  # nm
lj_eps = 0.99542  # AMU nm^2 ps^-2 = kJ mol^-1


acceleration_0 = calculate_acceleration(positions, mass, lj_sigma, lj_eps, box_len)
open("lennard_jones.xyz", "w").close()  # clear the file
for i in range(num_time_steps):
    if i % 5 == 0:
        # dump coordinates every 10 fs
        wrapped = wrap_coordinates(positions, box_len)
        write_xyz(wrapped, velocities, mass, box_len, "lennard_jones.xyz")

    # velocity verlet
    positions = next_positions(positions, velocities, acceleration_0, time_step)
    acceleration_1 = calculate_acceleration(positions, mass, lj_sigma, lj_eps, box_len)
    velocities = next_velocities(velocities, acceleration_0, acceleration_1, time_step)
    acceleration_0 = acceleration_1
```

- To advance the simulation forward we will be using the velocity Verlet algorithm.
- This is similar to the Verlet algorithm but updates both positions and velocities for each time step instead of just positions. This is useful because we will dump the velocities.

$$\mathbf{r}_i(t_{n+1}) = \mathbf{r}_i(t_n) + \mathbf{v}_i(t_n)\Delta t + \frac{1}{2}\mathbf{a}_i(t_n)\Delta t^2$$

$$t_{n+1} = t_n + \Delta t$$

$$\mathbf{v}_i(t_{n+1}) = \mathbf{v}_i(t_n) + \frac{1}{2}[\mathbf{a}_i(t_n) + \mathbf{a}_i(t_{n+1})]\Delta t$$

4. Lennard-Jones

```python
def next_positions(pos: np.ndarray, vels: np.ndarray, acc: np.ndarray, ds: float) -> np.ndarray:
    """Returns the next positions.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    acc : np.ndarray
        Acceleration of the atoms in the simulation.
    ds : float
        The time step.

    Returns
    -------
    np.ndarray
        The next position.
    """
    pass
```

```python
def next_velocities(vels: np.ndarray, acc_0: np.ndarray, acc_1: np.ndarray, ds: float):
    """Returns the next velocities.

    Parameters
    ----------
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    acc_0 : np.ndarray
        Acceleration of the atoms in the simulation at the current time.
    acc_1 : np.ndarray
        Acceleration of the atoms in the simulation at the next time step.
    ds : float
        The time step.

    Returns
    -------
    np.ndarray
        The next velocities.
    """
    pass
```

- To advance the simulation forward we will be using the velocity Verlet algorithm.
- This is similar to the Verlet algorithm but updates both positions and velocities for each time step instead of just positions. This is useful because we will dump the velocities.

$$\mathbf{r}_i(t_{n+1}) = \mathbf{r}_i(t_n) + \mathbf{v}_i(t_n)\Delta t + \frac{1}{2}\mathbf{a}_i(t_n)\Delta t^2$$

$$t_{n+1} = t_n + \Delta t$$

$$\mathbf{v}_i(t_{n+1}) = \mathbf{v}_i(t_n) + \frac{1}{2}[\mathbf{a}_i(t_n) + \mathbf{a}_i(t_{n+1})]\Delta t$$

- Position and velocities update function will be slightly different to the previous exercise.

4. Lennard-Jones

```python
def calculate_acceleration(  2 usages
        pos: np.ndarray, mass: float, sigma: float, eps: float, b_len: float):
    """Returns the acceleration on each atom. Forces are calculated
    between atom up to a cutoff of one half the box length.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    mass : float
        Mass of the atoms in the simulation.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_len : float
        The simulation box length.

    Returns
    -------
    np.ndarray
        Acceleration on each atom.
    """
    # diff = pos[:, None, :] - pos[None, :, :]  # big array of separation vectors
    # minimum image convention on diff
    # calculate distance matrix (r2)
    # np.fill_diagonal(r2, np.inf)  # avoids self interaction
    # r2[r2 > (b_len / 2) ** 2] = np.inf  # removes interactions beyond cutoff
    # calculate the forces
    pass
```

- The potential energy between Lennard-Jones atoms is:

$$V(\mathbf{r}_i, \mathbf{r}_j) = 4\varepsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^{6} \right]$$

- The force on atom i due to atom j can be obtained via a derivative of the potential energy.

$$F_{x_i} = -\frac{\partial V}{\partial x_i} = 24\varepsilon \left[2 \left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^{6} \right] \frac{x_i - x_j}{r_{ij}^2}$$

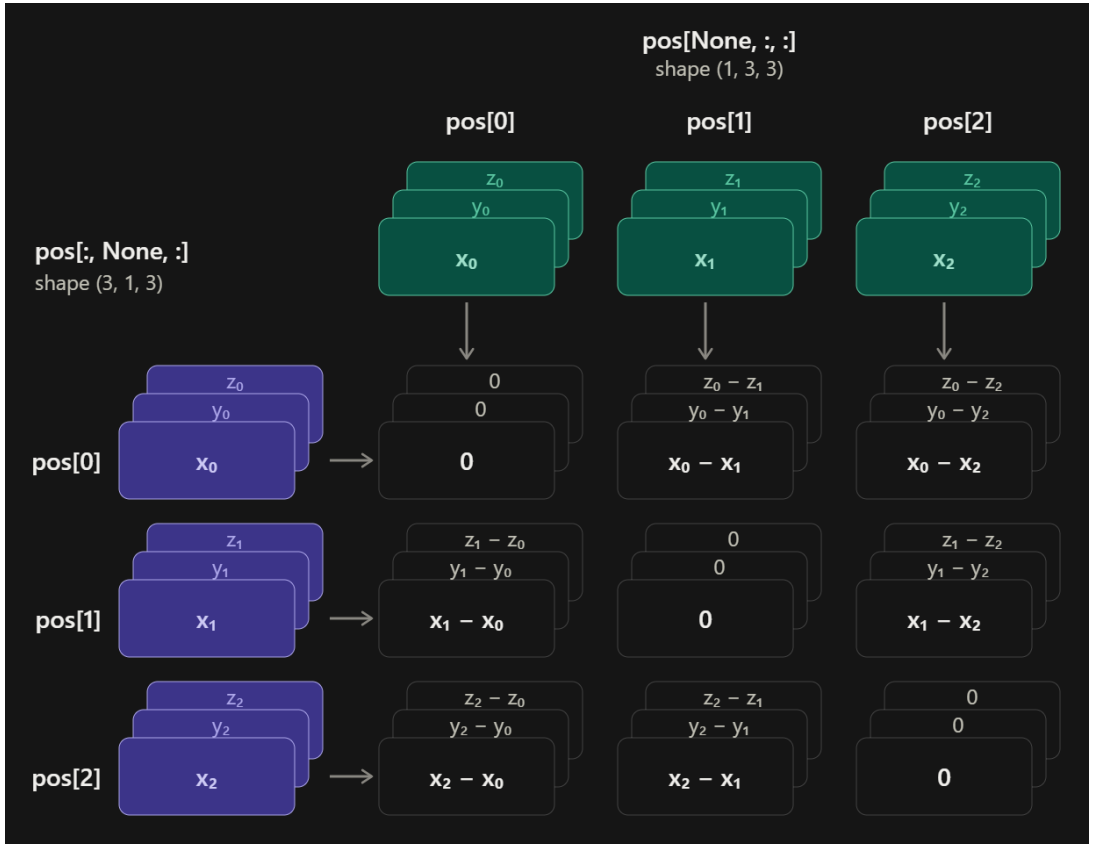
4. Lennard-Jones

```python
def calculate_acceleration(  2 usages
        pos: np.ndarray, mass: float, sigma: float, eps: float, b_len: float):
    """Returns the acceleration on each atom. Forces are calculated
    between atom up to a cutoff of one half the box length.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    mass : float
        Mass of the atoms in the simulation.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_len : float
        The simulation box length.

    Returns
    -------
    np.ndarray
        Acceleration on each atom.
    """
    # diff = pos[:, None, :] - pos[None, :, :]  # big array of separation vectors
    # minimum image convention on diff
    # calculate distance matrix (r2)
    # np.fill_diagonal(r2, np.inf)  # avoids self interaction
    # r2[r2 > (b_len / 2) ** 2] = np.inf  # removes interactions beyond cutoff
    # calculate the forces
    pass
```

- In the first line we calculate the displacement vectors between all atoms in one giant 3D array.



4. Lennard-Jones

```python
def calculate_acceleration(  2 usages
        pos: np.ndarray, mass: float, sigma: float, eps: float, b_len: float):
    """Returns the acceleration on each atom. Forces are calculated
    between atom up to a cutoff of one half the box length.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    mass : float
        Mass of the atoms in the simulation.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_len : float
        The simulation box length.

    Returns
    -------
    np.ndarray
        Acceleration on each atom.
    """
    # diff = pos[:, None, :] - pos[None, :                ay of separation vectors
    # minimum image convention on diff
    # calculate distance matrix (r2)
    # np.fill_diagonal(r2, np.inf)  # avoids self interaction
    # r2[r2 > (b_len / 2) ** 2] = np.inf  # removes interactions beyond cutoff
    # calculate the forces
    pass
```

- To calculate the force on the atom as if it were in a macroscopic system, we calculate the force acting on an atom in our MD simulation due to all periodic images up to some point e.g. cutoff radius, Ewald summation convergence criteria, or etc.
- We will do the simplest thing and calculate all interaction between atoms up to a cutoff of one half of the box size.
- When doing this we must make sure we calculate the force acting on an atom due to the closest periodic image of the other atom otherwise we may underestimate the result.

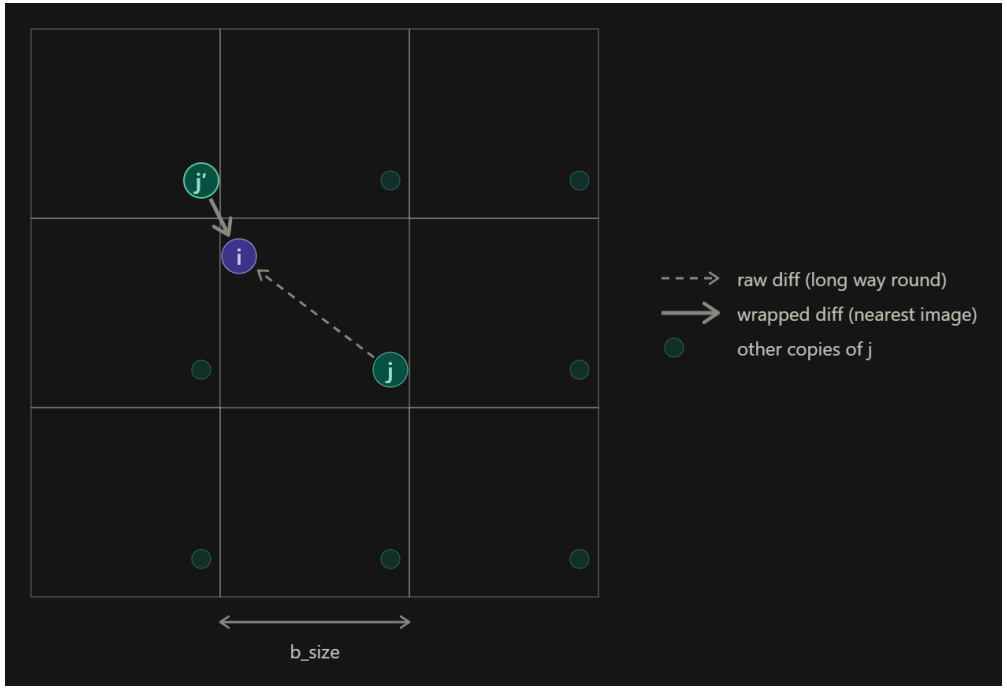
4. Lennard-Jones

```python
def calculate_acceleration(  2 usages
        pos: np.ndarray, mass: float, sigma: float, eps: float, b_len: float):
    """Returns the acceleration on each atom. Forces are calculated
    between atom up to a cutoff of one half the box length.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    mass : float
        Mass of the atoms in the simulation.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_len : float
        The simulation box length.

    Returns
    -------
    np.ndarray
        Acceleration on each atom.
    """
    # diff = pos[:, None, :] - pos[None, :                ay of separation vectors
    # minimum image convention on diff
    # calculate distance matrix (r2)
    # np.fill_diagonal(r2, np.inf)  # avoids self interaction
    # r2[r2 > (b_len / 2) ** 2] = np.inf  # removes interactions beyond cutoff
    # calculate the forces
    pass
```

- Minimum image convention. For the pair of atom images i and j we use the pair of images that are closest to each other.
- In our case want to calculate the Lennard-Jones force between the closest pairs so we can use the minimum image convention to obtain this.



4. Lennard-Jones

```
119    def calculate_acceleration(  2 usages
120            pos: np.ndarray, mass: float, sigma: float, eps: float, b_len: float):
121        """Returns the acceleration on each atom. Forces are calculated
122        between atom up to a cutoff of one half the box length.
123
124        Parameters
125        ----------
126        pos : np.ndarray
127            Positions of the atoms in the simulation.
128        mass : float
129            Mass of the atoms in the simulation.
130        sigma : float
131            Lennard-Jones sigma parameter.
132        eps : float
133            Lennard-Jones epsilon parameter.
134        b_len : float
135            The simulation box length.
136
137        Returns
138        -------
139        np.ndarray
140            Acceleration on each atom.
141        """
142        # diff = pos[:, None, :] - pos[None, :, :]  # big array of separati      tors
143        # minimum image convention on diff
144        # calculate distance matrix (r2)
145        # np.fill_diagonal(r2, np.inf)  # avoids self interaction
146        # r2[r2 > (b_len / 2) ** 2] = np.inf  # removes interactions beyond cutoff
147        # calculate the forces
148        pass
```

- When we did the displacement vector calculation remember we also calculated the distance between atoms with itself.
- Our distance matrix includes zeros with can set this to infinity to remove its contribution to the force calculation.

4. Lennard-Jones

```python
def calculate_acceleration(  2 usages
        pos: np.ndarray, mass: float, sigma: float, eps: float, b_len: float):
    """Returns the acceleration on each atom. Forces are calculated
    between atom up to a cutoff of one half the box length.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    mass : float
        Mass of the atoms in the simulation.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_len : float
        The simulation box length.

    Returns
    -------
    np.ndarray
        Acceleration on each atom.
    """
    # diff = pos[:, None, :] - pos[None, :, :]  # big array of separation vectors
    # minimum image convention on diff
    # calculate distance matrix (r2)
    # np.fill_diagonal(r2, np.inf)  # avoids self interaction
    # r2[r2 > (b_len / 2) ** 2] = np.inf  # removes interactions beyond cutoff
    # calculate the forces
    pass
```

- When we did the displacement vector calculation remember we also calculated the distance between atoms with itself.
- Our distance matrix includes zeros with can set this to infinity to remove its contribution to the force calculation.
- Similarly, we can remove contributions for distances greater the cutoff be also setting these to infinity.

4. Lennard-Jones

```python
def calculate_acceleration(  2 usages
        pos: np.ndarray, mass: float, sigma: float, eps: float, b_len: float):
    """Returns the acceleration on each atom. Forces are calculated
    between atom up to a cutoff of one half the box length.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    mass : float
        Mass of the atoms in the simulation.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_len : float
        The simulation box length.

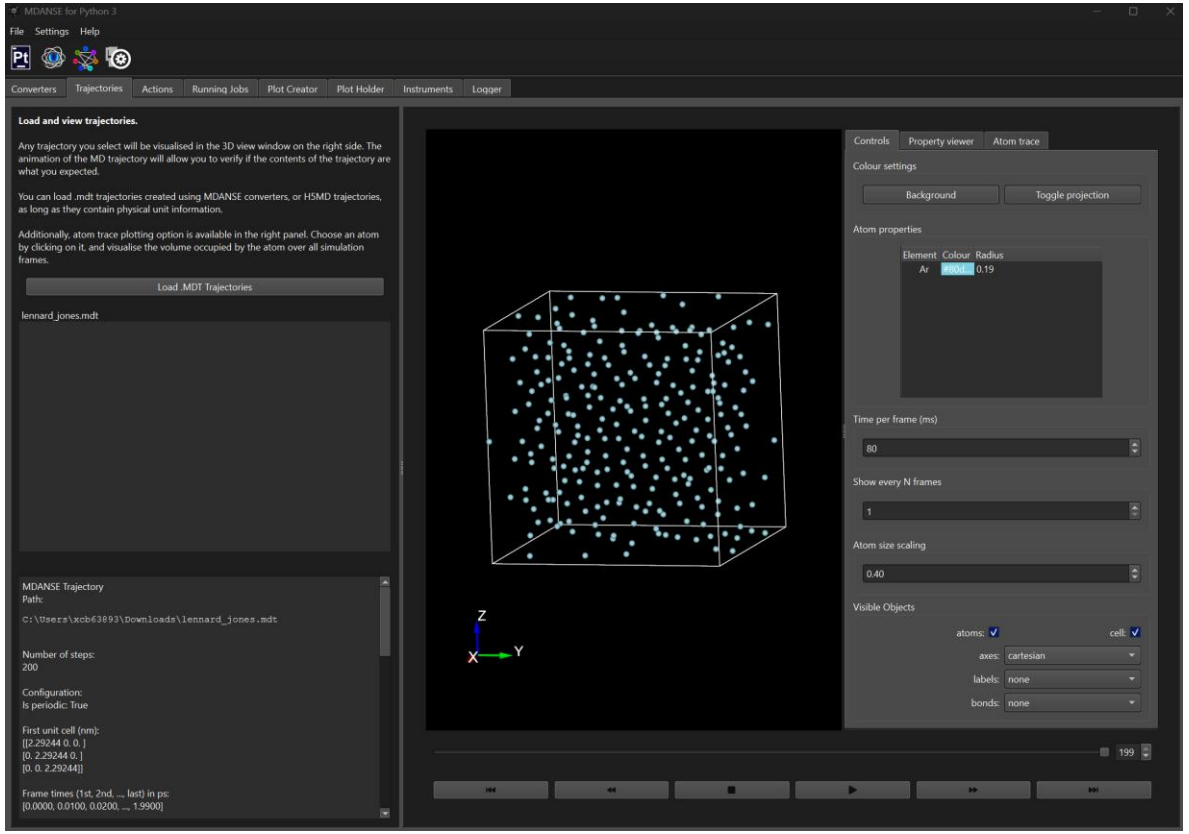
    Returns
    -------
    np.ndarray
        Acceleration on each atom.
    """
    # diff = pos[:, None, :] - pos[None, :, :]  # big array of separation vectors
    # minimum image convention on diff
    # calculate distance matrix (r2)
    # np.fill_diagonal(r2, np.inf)                self interaction
    # r2[r2 > (b_len / 2) **                    emoves interactions beyond cutoff
    # calculate the forces
    pass
```

- When we did the displacement vector calculation remember we also calculated the distance between atoms with itself.
- Our distance matrix includes zeros with can set this to infinity to remove its contribution to the force calculation.
- Similarly, we can remove contributions for distances greater the cutoff be also setting these to infinity.
- Finally, we can calculate the force on each atom and then return its acceleration.

$$F_{x_i} = -\frac{\partial V}{\partial x_i} = 24\varepsilon \left[2 \left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^{6} \right] \frac{x_i - x_j}{r_{ij}^2}$$

4. Lennard-Jones

- If you have successfully finished this example, try converting the file using the ASE converter in MDANSE to visualise it.
- If successful set the number of steps to 5000 or more to create a longer trajectory.
- We set the temperature to 85K when setting the velocities. Try running the temperature calculation. Can you explain what you see?



5. Energies

- In the last exercise we saw the temperature of the system change significantly as the MD simulation progressed.
- Let's try to understand this some more by calculating the potential and kinetic energies of the system.
- Fill in the missing code and then run the script to plot the potential, kinetic, and total energies.

```python
def potential_energy(  1 usage
        pos: np.ndarray, sigma: float, eps: float, b_len: float):
    """Returns the potential energy per atom of the system.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_len : float
        The simulation box length.

    Returns
    -------
    float
        The potential energy per atom.
    """
    # similar to the force calculation
    pass
```

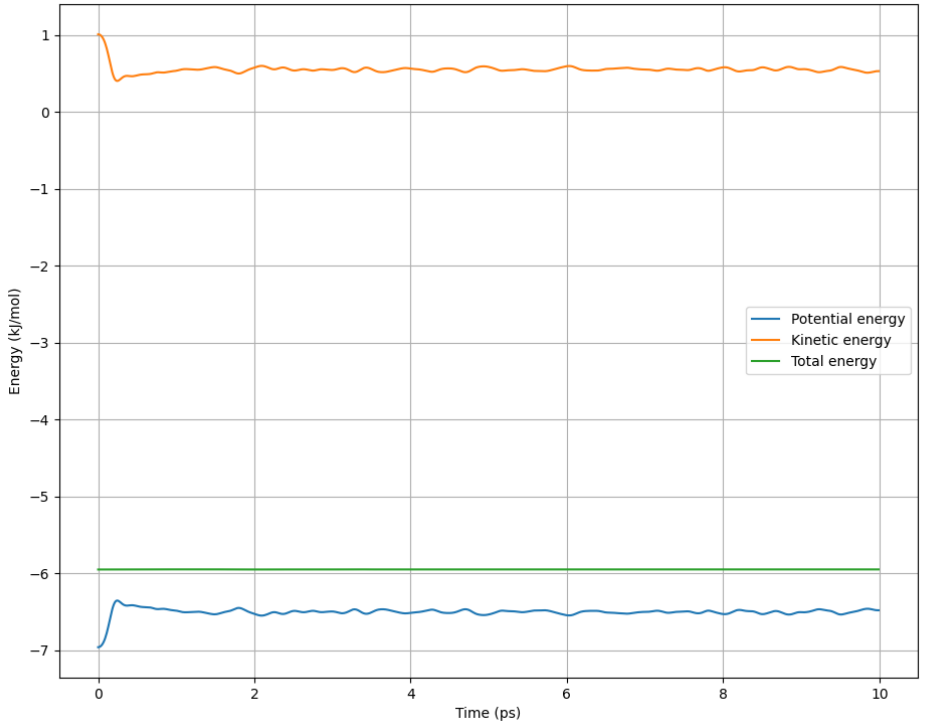
```python
def kinetic_energy(vels: np.ndarray, mass: float):  1 usage
    """Returns the kinetic energy per atom of the system.

    Parameters
    ----------
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    mass : float
        Mass of the atoms in the simulation.

    Returns
    -------
    float
        The kinetic energy per atom.
    """
    pass
```

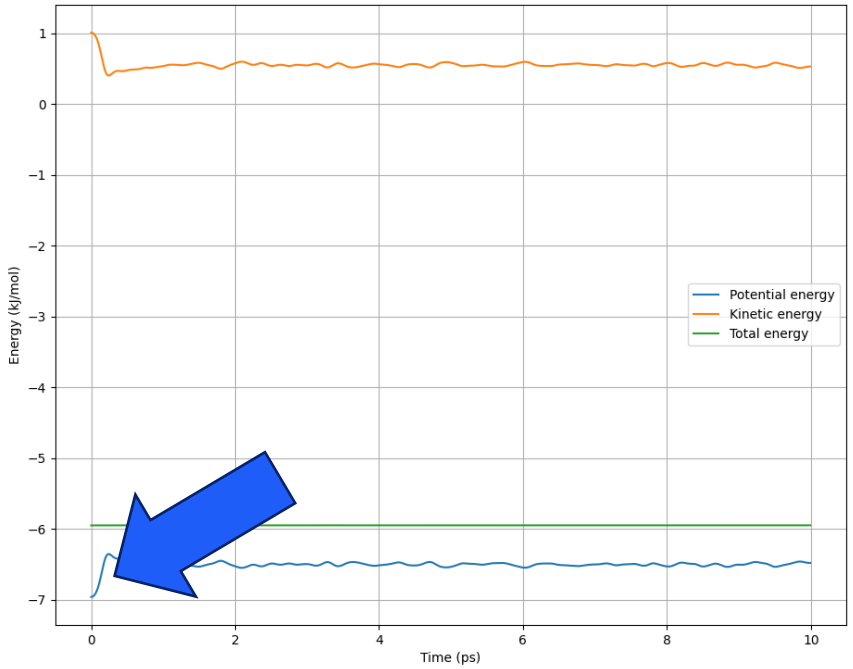
5. Energies

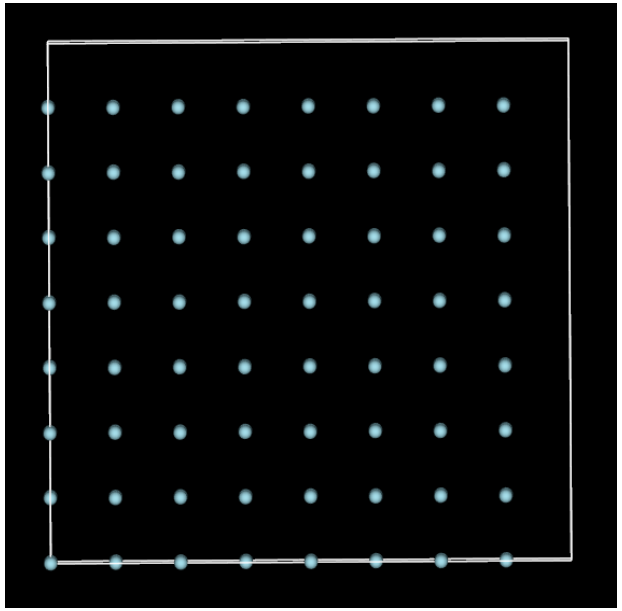
- If you have done this correctly you should see a plot similar to the one on the right.



5. Energies

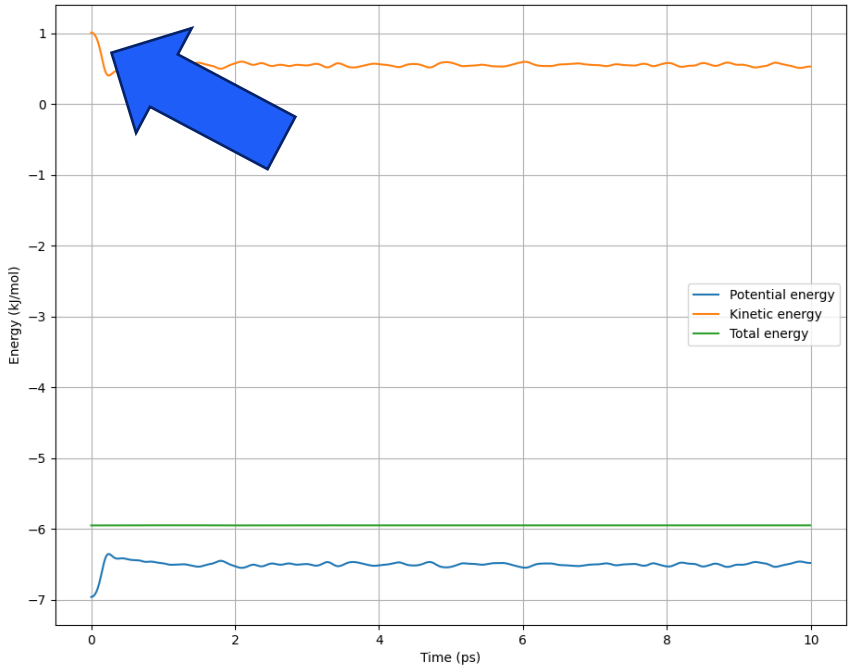
- If you have done this correctly you should see a plot similar to the one on the right.
- The initial positions placed argons atom in specific positions to form a FFC lattice. According to the plot we can see that this was a lower energy configuration.

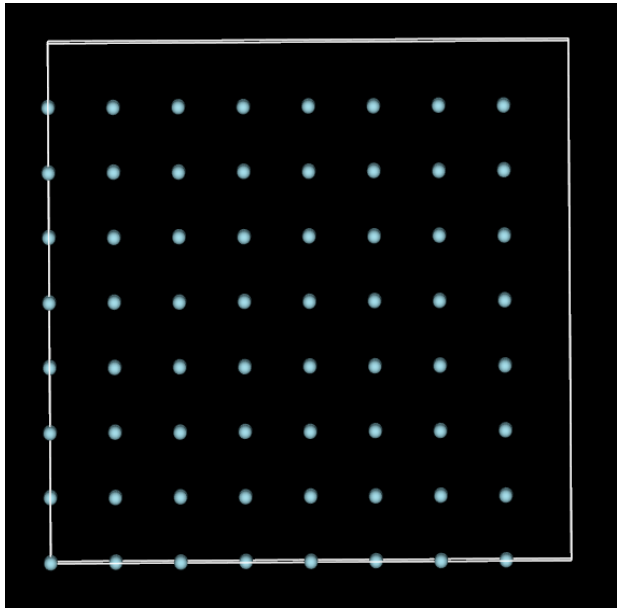




5. Energies

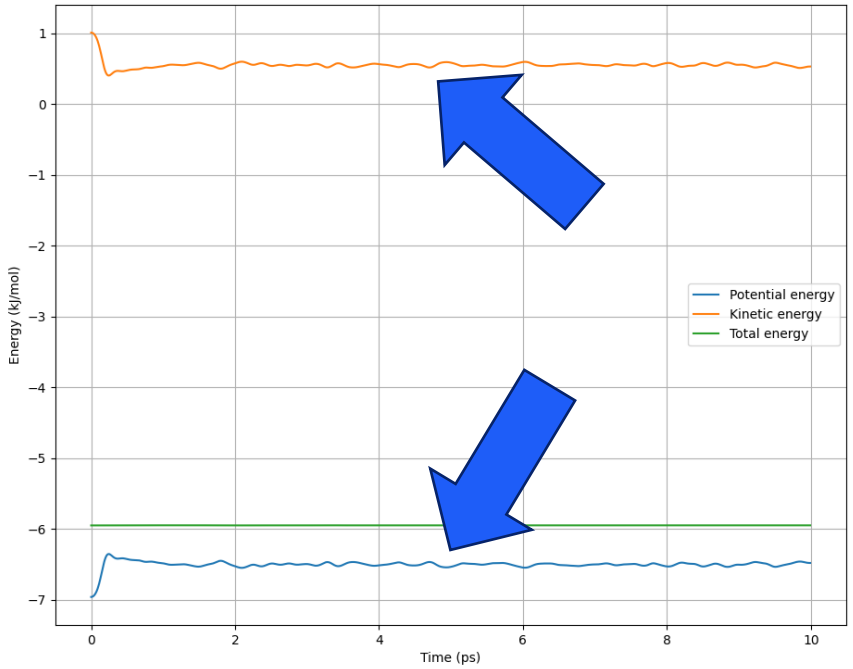
- If you have done this correctly you should see a plot similar to the one on the right.
- The initial positions placed argons atom in specific positions to form a FFC lattice. According to the plot we can see that this was a lower energy configuration.
- The velocities were set based on the Maxwell-Boltzmann distribution for a specific target temperature.

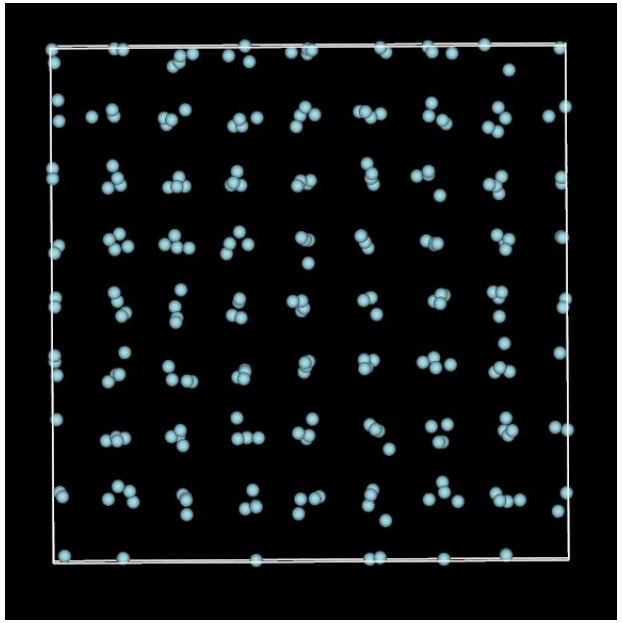




5. Energies

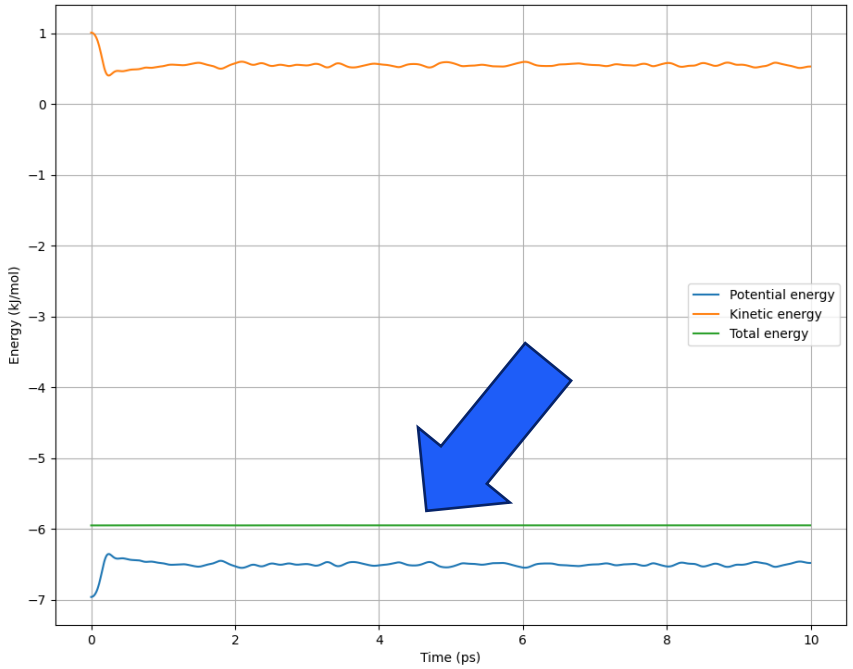
- As we advance the simulation there is an initial redistribution between the kinetic and potential and then reaches equilibrium as the kinetic and potential energies stabilise.

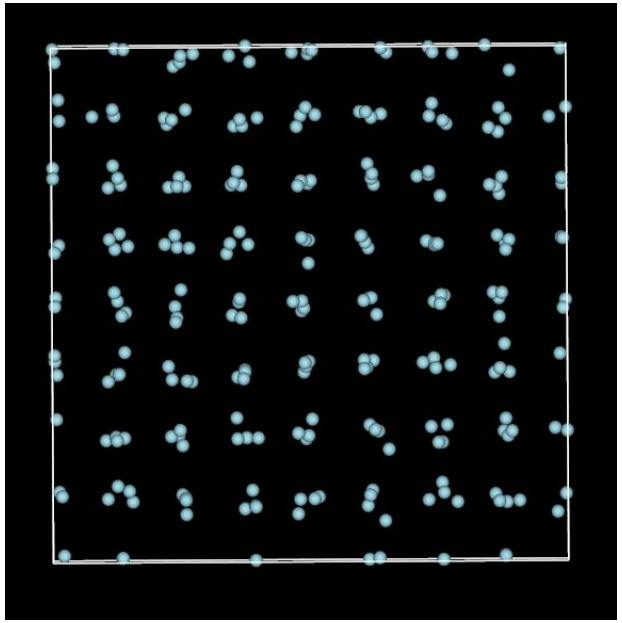




5. Energies

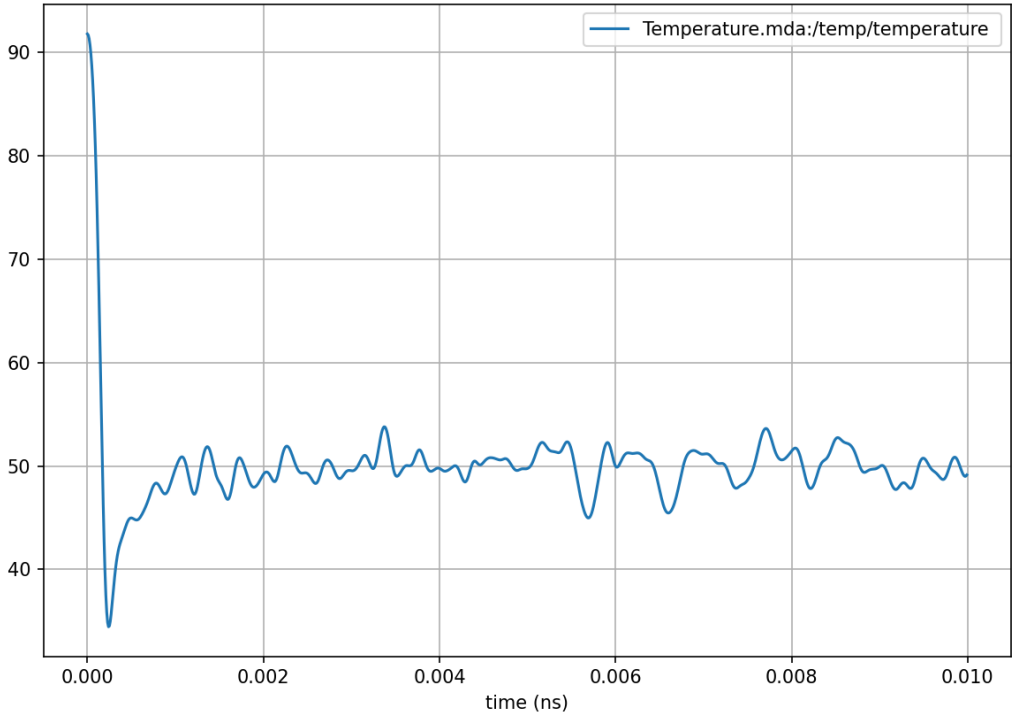
- As we advance the simulation there is an initial redistribution between the kinetic and potential and then reaches equilibrium as the kinetic and potential energies stabilise.
- Notice that throughout the simulation the total energy was constant (plus some noise).





5. Energies

- As we advance the simulation there is an initial redistribution between the kinetic and potential and then reaches equilibrium as the kinetic and potential energies stabilise.
- Notice that throughout the simulation the total energy was constant (plus some noise).
- The initial change in the kinetic energy and therefore temperature is problematic since we might want to calculate properties at a specific temperature.



6. Temperature



- In the previous example we ran a simulation and generated trajectories at constant energy.
- When calculating results numerically from that MD simulation generally we want to do this at constant number of particles, volume and energy (NVE). Especially if we are calculating dynamics results from MD.
- However, we may want to equilibrate the simulation at a specific temperature to match some experiment before switching back to an NVE simulation for production.
- To do this requires a thermostat we will look at the Langevin thermostat.

6. Temperature

- We will follow the algorithm Leap-Frog middle algorithm as used in OpenMM [1, 2].
- The basic equations are as follows:

$$\mathbf{v}_i(t_{n+1/2}) = \mathbf{v}_i(t_{n-1/2}) + \mathbf{a}_i(t_n)\Delta t$$

$$\mathbf{r}_i(t_{n+1/2}) = \mathbf{r}_i(t_n) + \frac{1}{2}\mathbf{v}_i(t_{n+1/2})\Delta t$$

$$\mathbf{v}_i(t_{n+1/2}) \rightarrow \mathbf{v}'_i(t_{n+1/2}) \qquad \text{thermostat}$$

$$\mathbf{r}_i(t_{n+1}) = \mathbf{r}_i(t_{n+1/2}) + \frac{1}{2}\mathbf{v}'_i(t_{n+1/2})\Delta t$$



[1] https://docs.openmm.org/latest/userguide/theory/04_integrators.html
[2] Zhijun Zhang, Xinzijian Liu, Kangyu Yan, Mark E. Tuckerman, Jian Liu; Unified Efficient Thermostat Scheme for the Canonical Ensemble with Holonomic or Isokinetic Constraints via Molecular Dynamics. *J. Phys. Chem. A* 18 July 2019; 123 (28): 6056–6079.

6. Temperature

```python
acceleration = calculate_acceleration(positions, mass, lj_sigma, lj_eps, box_len)
velocities = next_velocities(velocities, acceleration, -time_step / 2)
open("temperature.xyz", "w").close()  # clear the file
for i in range(num_time_steps):
    acceleration = calculate_acceleration(positions, mass, lj_sigma, lj_eps, box_len)

    if i % 5 == 0:
        # dump coordinates every 10 fs
        wrapped = wrap_coordinates(positions, box_len)
        vel_for_dump = next_velocities(velocities, acceleration, time_step / 2)
        write_xyz(wrapped, vel_for_dump, mass, box_len, "temperature.xyz")

    # Leap-🐸 middle
    velocities = next_velocities(velocities, acceleration, time_step)
    positions = next_positions(positions, velocities, time_step / 2)
    velocities = thermostat_step(velocities, temperature, friction, mass, time_step)
    positions = next_positions(positions, velocities, time_step / 2)
```

```python
def thermostat_step(vels: np.ndarray, temp: float, fric: float, mass: float, ds: float):
    """Applies the thermostat to the input velocities.

    Parameters
    ----------
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    temp : float
        The temperature.
    fric: float
        The friction coefficient.
    mass : float
        The mass of the atoms
    ds: float
        The time step

    Returns
    -------
    np.ndarray
        The velocit                    hermostat applied to them.
    """
    return vels
```

- Let's start by redoing the Lennard-Jones system but with the Leap-Frog middle integration scheme.
- First let's do it without the thermostat update step. We just simply return the normal velocities.

$$\mathbf{v}_i(t_{n+1/2}) = \mathbf{v}_i(t_{n-1/2}) + \mathbf{a}_i(t_n)\Delta t$$

$$\mathbf{r}_i(t_{n+1/2}) = \mathbf{r}_i(t_n) + \frac{1}{2}\mathbf{v}_i(t_{n+1/2})\Delta t$$

$$\mathbf{r}_i(t_{n+1}) = \mathbf{r}_i(t_{n+1/2}) + \frac{1}{2}\mathbf{v}'_i(t_{n+1/2})\Delta t$$

6. Temperature

```python
def next_positions(pos: np.ndarray, vels: np.ndarray, ds: float) -> np.ndarray:
    """Advances the positions forward one half of a time step.

    Parameters
    ----------
    pos : np.ndarray
        Positions of the atoms in the simulation.
    vels : np.ndarray
        Velocities of the atoms in the simulation at the current
        time plus one half of a time step.
    ds : float
        The time step.

    Returns
    -------
    np.ndarray
        The next position.
    """
    pass
```

```python
def next_velocities(vels: np.ndarray, acc: np.ndarray, ds: float):  2 usages
    """Advances the velocities forward one time step.

    Parameters
    ----------
    vels : np.ndarray
        Velocities of the atoms in the simulation at the current
        time minus one half of a time step.
    acc : np.ndarray
        Acceleration of the atoms in the simulation at the current time.
    ds : float
        The time step.

    Returns
    -------
    np.ndarray
        The next velocities.
    """
    pass
```

- Let's start by redoing the Lennard-Jones system but with the Leap-Frog middle integration scheme.
- First let's do it without the thermostat update step. We just simply return the normal velocities.
- Update the positions and velocity functions then check it runs fine.

$$\mathbf{v}_i(t_{n+1/2}) = \mathbf{v}_i(t_{n-1/2}) + \mathbf{a}_i(t_n)\Delta t$$

$$\mathbf{r}_i(t_{n+1/2}) = \mathbf{r}_i(t_n) + \frac{1}{2}\mathbf{v}_i(t_{n+1/2})\Delta t$$

$$\mathbf{r}_i(t_{n+1}) = \mathbf{r}_i(t_{n+1/2}) + \frac{1}{2}\mathbf{v}'_i(t_{n+1/2})\Delta t$$

6. Temperature

$$\mathbf{v}_i'(t_n) = \mathbf{v}_i(t_n)\alpha + \mathbf{R}\sqrt{k_\mathrm{B}T(1-\alpha^2)/m}$$

$$\alpha = \exp(-\gamma \Delta t)$$

$$R_x \sim \mathcal{N}(0,1)$$

```python
def thermostat_step(vels: np.ndarray, temp: float, fric: float, mass: float, ds: float):
    """Applies the thermostat to the input velocities.

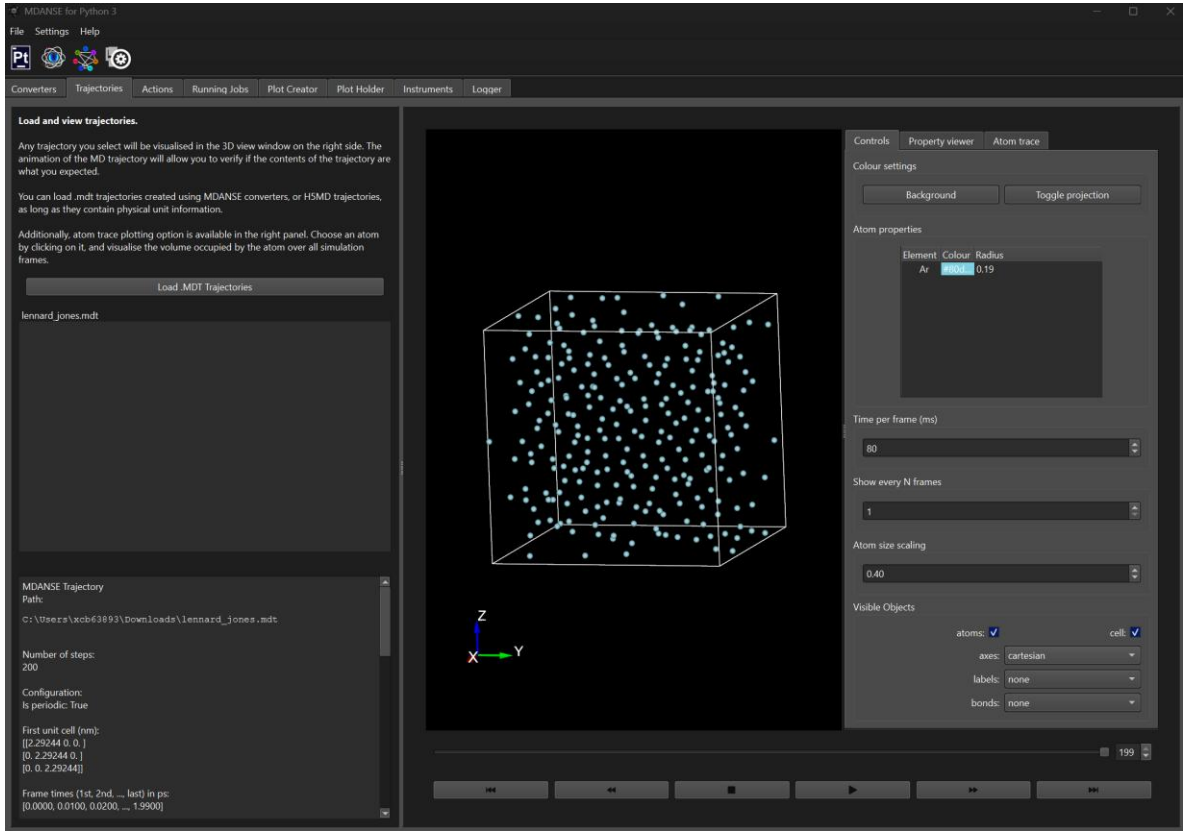
    Parameters
    ----------
    vels : np.ndarray
        Velocities of the atoms in the simulation.
    temp : float
        The temperature.
    fric: float
        The friction coefficient.
    mass : float
        The mass of the atoms
    ds: float
        The time step

    Returns
    -------
    np.ndarray
        The velocities with the thermostat applied to them.
    """
    return vels
```

- Now let's implement the Langevin thermostat.
- γ is the friction coefficient, and R is a random number drawn from a normal distribution.
- So, for every iteration every atom in our trajectory its velocities are given a random kick.

6. Temperature

- If you have successfully finished this example, try converting the file using the ASE converter in MDANSE to visualise it.
- If successful set the number of steps to 5000 or more to create a longer trajectory. Then run a temperature calculation.
- Try changing the temperature. Can you melt the structure?
- Using code from the `4_energies.py` script can you plot the energies? Can you explain what you see?



7. Pressure



- In the previous example we ran a simulation and generated trajectories at a fixed volume.
- We may want to equilibrate the simulation at a specific pressure to match some experiment before switching back to an NVT and then NVE simulation for production.
- To do this requires a barostat we will look at the Monte Carlo barostat.

7. Pressure

```
299     acceleration = calculate_acceleration(positions, mass, lj_sigma, lj_eps, box_len)
300     velocities = next_velocities(velocities, acceleration, -time_step / 2)
301     open("pressure.xyz", "w").close()  # clear the file
302     for i in range(num_time_steps):
303         acceleration = calculate_acceleration(positions, mass, lj_sigma, lj_eps, box_len)
304
305         if i % 5 == 0:
306             # dump coordinates every 10 fs
307             wrapped = wrap_coordinates(positions, box_len)
308             vel_for_dump = next_velocities(velocities, acceleration, time_step / 2)
309             write_xyz(wrapped, vel_for_dump, mass, box_len, "pressure.xyz")
310
311         # Leap-🐸 middle
312         velocities = next_velocities(velocities, acceleration, time_step)
313         positions = next_positions(positions, velocities, time_step / 2)
314         velocities = thermostat_step(
                                      cities, temperature, friction, mass, time_step)
315         positions = next_posi
                                      ns, velocities, time_step / 2)
316
317         if i % 20 == 0:
318             # apply the montecarlo barostat every 20 steps
319             positions, box_len = barostat_step(
320                 positions, pressure, temperature, lj_sigma, lj_eps, box_len)
```

- We will add a Monte Carlo Barostat which changes the volume of our simulation [1,2,3] depending on the target temperature and pressure.

[1] https://docs.openmm.org/7.1.0/userguide/theory.html#montecarlobarostat
[2] https://doi.org/10.1016/j.cplett.2003.12.039
[3] https://doi.org/10.1016/0010-4655(95)00059-O

7. Pressure

```python
def barostat_step(  1 usage
        pos_0: np.ndarray, press: float, temp: float, sigma: float,
        eps: float, b_0: float, dv_max: float
):
    """Monte Carlo barostat.

    Parameters
    ----------
    pos_0 : np.ndarray
        Positions of the atoms in the simulation.
    press : float
        The pressure.
    temp : float
        The temperature.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_0 : float
        The simulation box length.
    dv_max : float
        The max volume change.


    Returns
    -------
    tuple[np.ndarray, float]
        The new positions and box length.
    """
    # n_atms = pos_0.shape[0]
    # box_vol = b_0**3

    # tries a volume change of dv_max, usuall  this  actively changed/updated
    # so that the overall move acceptance ra  , but we will skip
    # this as it a dv_max change works fine  the extra coding
    # del_v = dv_max * rng.uniform(-1, 1)
    pass
```

- Here we will need to implement the Monte Carlo barostat. To make it easy and set it so that we only make volume changes to up a max of 1% of the current box volume.
- Usually, this scaling factor is actively changed over the simulation so that a specific MC move acceptance rate is obtain 50% or so.

7. Pressure

```python
def barostat_step(  1 usage
        pos_0: np.ndarray, press: float, temp: float, sigma: float,
        eps: float, b_0: float, dv_max: float
):
    """Monte Carlo barostat.

    Parameters
    ----------
    pos_0 : np.ndarray
        Positions of the atoms in the simulation.
    press : float
        The pressure.
    temp : float
        The temperature.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_0 : float
        The simulation box length.
    dv_max : float
        The max volume change.


    Returns
    -------
    tuple[np.ndarray, float]
        The new positions and box length.
    """
    # n_atms = pos_0.shape[0]
    # box_vol = b_0**3

    # tries a volume change of dv_max, usually this  actively changed/updated
    # so that the overall move acceptance ra , but we will skip
    # this as it a dv_max change works fine  the extra coding
    # del_v = dv_max * rng.uniform(-1, 1)
    pass
```

- Here we will need to implement the Monte Carlo barostat. To make it easy and set it so that we only make volume changes to up a max of 1% of the current box volume.
- Usually, this scaling factor is actively change over the simulation so that a specific MC move acceptance rate is obtain 50% or so.
- In Monte Carlo methods we most first create the trial configuration that we want to move to. First, we randomly determine a volume change to make.

$$\Delta V = Ar$$

Scaling factor

7. Pressure

```python
def barostat_step(  1 usage
        pos_0: np.ndarray, press: float, temp: float, sigma: float,
        eps: float, b_0: float, dv_max: float
):
    """Monte Carlo barostat.

    Parameters
    ----------
    pos_0 : np.ndarray
        Positions of the atoms in the simulation.
    press : float
        The pressure.
    temp : float
        The temperature.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_0 : float
        The simulation box length.
    dv_max : float
        The max volume change.


    Returns
    -------
    tuple[np.ndarray, float]
        The new positions and box length.
    """
    # n_atms = pos_0.shape[0]
    # box_vol = b_0**3

    # tries a volume change of dv_max, usually this would be actively changed/updated
    # so that the overall move acceptance rate is around 50%, but we will skip
    # this as it a dv_max change works fine and avoids the extra coding
    # del_v = dv_max * rng.uniform(-1, 1)
    pass
```

- Here we will need to implement the Monte Carlo barostat. To make it easy and set it so that we only make volume changes to up a max of 1% of the current box volume.
- Usually, this scaling factor is actively change over the simulation so that a specific MC move acceptance rate is obtain 50% or so.
- In Monte Carlo methods we most first create the trial configuration that we want to move to. First, we randomly determine a volume change to make.

$$\Delta V = Ar$$

Random number between -1 and 1

Scaling factor

7. Pressure

```python
def barostat_step(  1 usage
        pos_0: np.ndarray, press: float, temp: float, sigma: float,
        eps: float, b_0: float, dv_max: float
):
    """Monte Carlo barostat.

    Parameters
    ----------
    pos_0 : np.ndarray
        Positions of the atoms in the simulation.
    press : float
        The pressure.
    temp : float
        The temperature.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_0 : float
        The simulation box length.
    dv_max : float
        The max volume change.


    Returns
    -------
    tuple[np.ndarray, float]
        The new positions and box length.
    """
    # n_atms = pos_0.shape[0]
    # box_vol = b_0**3

    # tries a volume change of dv_max, usually this would be actively changed/updated
    # so that the overall move acceptance rate is around 50%, but we will skip
    # this as it a dv_max change works fine and avoids the extra coding
    # del_v = dv_max * rng.uniform(-1, 1)
    pass
```

- Next, we determine the scaling factor to scale out box lengths and atom positions.

$$s = \left(\frac{V + \Delta V}{V} \right)^{1/3}$$

$$l' = sl \qquad \mathbf{r}' = s\mathbf{r}$$

7. Pressure

```python
def barostat_step(  1 usage
        pos_0: np.ndarray, press: float, temp: float, sigma: float,
        eps: float, b_0: float, dv_max: float
):
    """Monte Carlo barostat.

    Parameters
    ----------
    pos_0 : np.ndarray
        Positions of the atoms in the simulation.
    press : float
        The pressure.
    temp : float
        The temperature.
    sigma : float
        Lennard-Jones sigma parameter.
    eps : float
        Lennard-Jones epsilon parameter.
    b_0 : float
        The simulation box length.
    dv_max : float
        The max volume change.


    Returns
    -------
    tuple[np.ndarray, float]
        The new positions and box length.
    """
    # n_atms = pos_0.shape[0]
    # box_vol = b_0**3

    # tries a volume change of dv_max, usually this would be actively changed/updated
    # so that the overall move acceptance rate is around 50%, but we will skip
    # this as it a dv_max change works fine and avoids the extra coding
    # del_v = dv_max * rng.uniform(-1, 1)
    pass
```

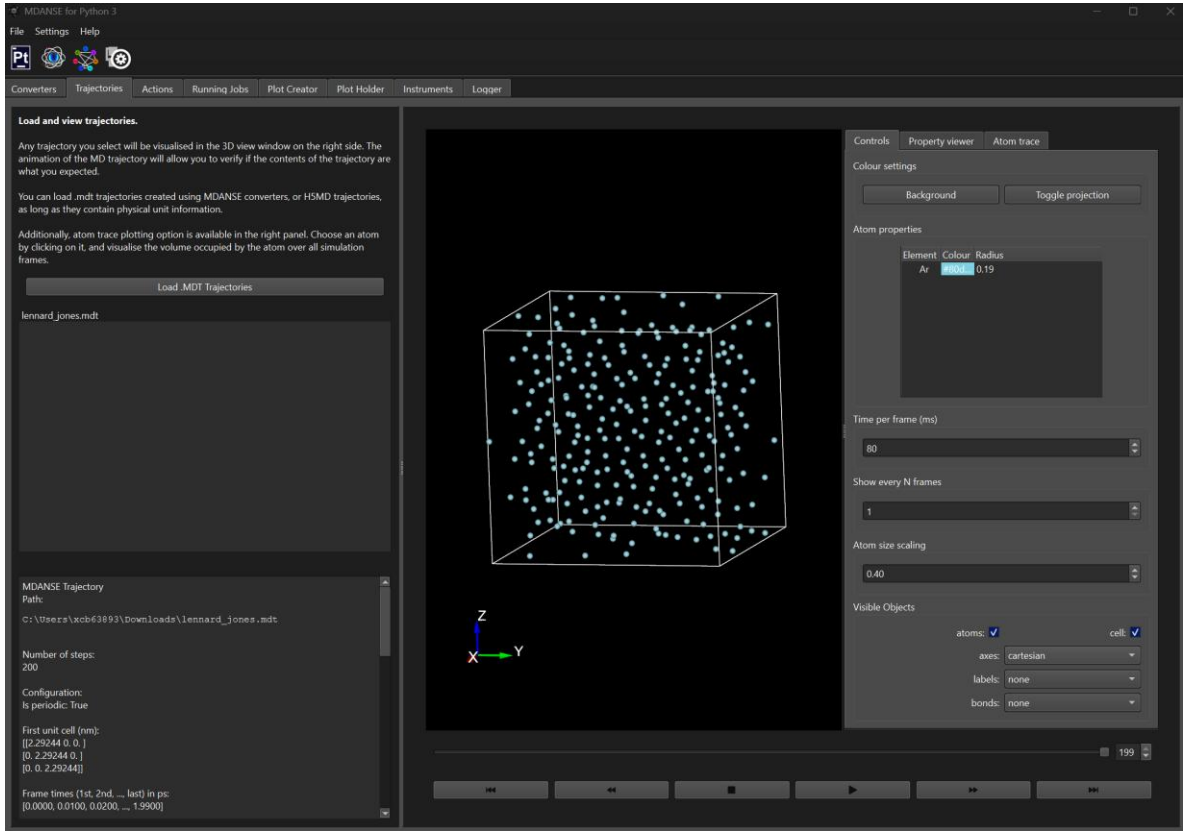
- Next, we determine the scaling factor to scale out box lengths and atom positions.
- Once we have our new configuration, we can now make our Monte Carlo move which is accepted based with the probability.

$$P(\Delta W) = \begin{cases} \exp\left(\frac{\Delta W}{k_{\mathrm{B}}T}\right) & \Delta W > 0 \\ 1 & \text{otherwise} \end{cases}$$

$$\Delta W = \Delta E + P\Delta V - Nk_{\mathrm{B}}T \ln\left(\frac{V + \Delta V}{V}\right)$$

7. Pressure

- If you have successfully finished this example, try converting the file using the ASE converter in MDANSE to visualise it.
- If successful set the number of steps to 10000 or more to create a longer trajectory.
- Try running the temperature and density calculations in MDANSE. Does your volume stabilised?
- Try increasing the temperature.



Molecular dynamics coding exercises

- Congratulations, you have completed MD coding exercises.
- You have worked from almost all the basic ideas behind MD simulations including periodic boundaries, integrators, thermostats, barostats, and force fields.
- To build on what you learnt, you can use these scripts to implement more features like other thermostats (Nose-Hoover chain), Barostats, or nearest neighbour lists.

